Applied Soft Computing 128 (2022) 109513

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Predicting population size and termination criteria in metaheuristics: A case study based on spotted hyena optimizer and crow search algorithm

Emanuel Vega [a,*], Ricardo Soto [a], Broderick Crawford [a], Javier Peña [a], Pablo Contreras [a], Carlos Castro [b]

[a] Pontificia Universidad Católica de Valparaíso, Valparaíso 2362807, Chile
[b] Universidad Técnica Federico Santa María, Valparaíso 2390123, Chile

ARTICLE INFO

Article history:
Received 25 January 2022
Received in revised form 21 July 2022
Accepted 10 August 2022
Available online 18 August 2022

Keywords:
Metaheuristics
Machine learning
Hybrid approach
Optimization

ABSTRACT

The proper configuration of a metaheuristic requires specific and advanced knowledge, for instance to decide an appropriate population size, to effectively set probability parameters, or to define a correct termination criterion. During the last years there is a trend to let the metaheuristic to autonomously self-control the internal configuration. In this way, by using learning mechanisms, the algorithm is able to learn from the solving process and automatically adapt its configuration in order to improve their performance. Following this research trend, in this paper we explore the ability to autonomously predict population size and termination criteria. The population prediction allows the metaheuristic to dynamically vary the population size during solving time. In this regard, a suitable number of agents is used for each part of the process in order to efficiently explore the space of solutions. On the other hand, predicting the termination criteria is useful as well, the smart usage of resources in the optimization field can bring several advantages in solving complex industrial problems. In this regard, since the number of iterations is estimated during solving time, the proposed architecture avoids useless processing time that does not lead to any solution improvement. We illustrate interesting results using the spotted hyena optimizer and crow search algorithm solving 24 benchmarks functions and the multidimensional knapsack problem, where the proposed approach notably competes with several state-of-the-art optimization algorithms.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Metaheuristics have been a hot topic for an extensive period of time mostly because several optimization problems have been born as a consequence of the evolution in the technology. Thus, in order to tackle hard optimization problems which are in constant evolution, the employment of approximated methods such as metaheuristics have been a recurrent answer. The main feature associated with these kinds of methods is being able to find possible optimum solutions in reasonable computational resources. In this context, multiple methods and several taxonomies have been proposed, such as single-solution, nature-inspired, population-based, and so on [1,2]. The popularity had such a deep impact in the field that the usage of these methods in a cooperative architecture became a new standard strategy in the design of novel approaches. In this regard, multiple interactions have been reported in the literature [3], such as optimization assisting optimization [4], optimization methods assisting Machine Learning (ML) [5], ML assisting ML [6,7], and ML assisting optimization [8]. The proposed work falls in the last classification, where the data-driven architecture performs under a population-based strategy where a finite amount of agents evolve while carrying out the search in the solution space. The search process is well-known for generating a large amount of dynamic data, for instance, feasible solutions, best solutions, bad solutions, local optima, etc. In this work, we exploit these output resources with the objective to generate knowledge in order to drive the search.

The design of the proposed architecture, named Learning-based Modular Balancer (LBMB), is composed of four modules. The first module, called Movements, is highly related to the metaheuristic field. The main objective is the management in the performance of intensification and diversification of the selected algorithm. In this attempt to design LBMB, we employ 2 different metaheuristic algorithms, the Spotted Hyena Optimizer (SHO)

* Corresponding author.
E-mail addresses: emanuel.vega.m@mail.pucv.cl (E. Vega), ricardo.soto@pucv.cl (R. Soto), broderick.crawford@pucv.cl (B. Crawford), javier.pena.r@mail.pucv.cl (J. Peña), pablo.contreras.e@mail.pucv.cl (P. Contreras), carlos.castro@inf.utfsm.cl (C. Castro).

https://doi.org/10.1016/j.asoc.2022.109513
1568-4946/© 2022 Elsevier B.V. All rights reserved.

algorithm [9] and the Crow search algorithm (CSA) [10]. The main feature of SHO comes from the ability to search in the solution space in a cluster-based fashion, which has proved to be a successful option solving optimization problems [11–13]. The main feature of CSA comes from the similarities between the behaviors of crows as an optimization process, which is known as swarm behaviors like searching for food, flying through distance searching for an objective, and so on [14]. The second module, named Administrator, includes the removal of agents from the population, adding new randomly generated agents to the population, and management of the size to perform at certain times while carrying out the search. In this regard, the sizes configured to perform are defined by the names of schemes, initially each scheme has equal probabilities to be selected and modify the amount of agents searching the solution space. However, the probabilities will be subject to modification influenced by the regression analysis associated with each scheme. Also, the mechanism designed for this component is a Monte Carlo probabilistic roulette. The third and four modules, named Learning-population and Learning-Termination criteria, make use of the dynamic data generated through the search by the population to influence over the previously mentioned module (Administrator) by the means of knowledge. The objective of the Learning-population consists of prediction of a possible better performance taking into account different performance-based metrics corresponding to each scheme selected, thus, the model performs as many regressions as scheme defined, the best computed prognostic value is given priority for the next scheme selection. The objective behind the Learning-Termination criteria, evaluate possible improvements in performance taking into consideration the amount of iterations to keep performing the search process in the solution space. We carry out the analysis by the means of multiple linear regressions, the metrics employed includes the percentage of feasible solutions, percentage of best solutions achieved, percentage of infeasible solutions, and the strategy/scheme employed.

In order to test the viability of our proposed approach, we carry out experimentation tests over two sets of well-known continuous benchmark functions, and detailed information about the results achieved are presented. In this work, the competitiveness is achieved by comparing results reported by state-of-the-art algorithms, classic versions of SHO and CSA, also, SHO and CSA assisted by IRace. Moreover, in order to further test the capabilities of LMPB, a set of instances from the multidimensional knapsack problem are solved. In this first attempt designing LBMB, we illustrate interesting and promising results. Thus, giving us optimistic motivations and ideas to further improve and test new thresholds of knowledge in the optimization field.

The main contributions can be illustrated as follows.

- Robust hybrid architecture to tackle optimization problems.
- A key issue in population-based approaches: Adapting population size on run-time.
- A key issue in metaheuristic approaches: Adapting the termination criteria on run-time.
- Scalability(module 1), movement operators from different algorithms can be employed.
- Scalability(modules 3, 4), incorporation of multiple machine learning methods.

The rest of this paper is organized as follows, the next section illustrates the related work. The details regarding the designed LBMB are presented in Section 3. Section 4 illustrates results achieved, detailed comparison, and discussion. Finally, we conclude and suggest some new lines of future work.

2. Related work

Currently, the inabilities from exact methods tackling hard optimization problems has given new opportunities to approximated and evolved methods to be exploited by the scientific community. Concerning approximated methods, metaheuristics have taken a bright spot in the last decades [15]. The generic nature defined in these types of methods demonstrated that they can be applied to several optimization problems. They usually follow defined design to surf the solution space, then incorporate multiple stochastic components, parameters, and movement operators in their solving strategy. In this context, as time goes by, improvements have been performed over these methods, such as being assisted by multiple optimization tools [16,17].

The on-line and off-line tuning of parameters is usually considered a task for experts, which is a common case for algorithms in the machine learning, metaheuristic, and many other optimization fields [18]. Moreover, it has been illustrated in [19], that 90% of the time implementing a new metaheuristic is specially dedicated to the tuning of parameters employed to perform the search. In this regard, a frequently used strategy includes the employment of external optimization tools in order to assist this tedious task. For instance, in [20], a complex Evolutionary Algorithm (EA) is designed, the main objective is to evolve a EA under the assistance of Genetic Algorithm (GA). Also, in [21], an online learning hyper-heuristic employing a data science technique is proposed. The idea is to employ tensor analysis in order to self-improve while tackling the nurse rostering problem.

This paper is concerned with the management of population size and the termination criteria for LBMB. In this regard, successful hybrids have been proposed in the literature tackling population. In [12,22], authors employ Autonomous Search (AS) as a reactive measurement action in order to modify the population when poor performance is detected. They test a small sample of agents from the population, when there are no better fitness values achieved over an specific amount of iterations, the population for Human behavior-based optimization (HBBO) and Spotted Hyena optimizer (SHO) are modified. Also, concerning learning-based population approaches, we can find multiple cluster-based strategies. Firstly, a k-means-based model is employed in order to redistribute the agents in each iteration, parameters are adapted based on the knowledge extracted in the clustering procedure, and probabilities are sorted out in EA operators [23]. Furthermore, Streichert et al. [24], proposed the clustering-based niching method with the objective to determine several global and local optima in a multimodal search space. The model was employed over well-known EAs, the aim of the model is to preserve diversity by the use of a multi population approach. Furthermore, data mining-based approaches to tackle the tuning of the population are illustrated in [25,26], which proposes hybrid approaches working over parameters of the GRASP metaheuristic.

On the other hand, the termination criteria is a key issue hardly improved under preliminary attempts in the design of an optimization method. Concerning preliminary common strategies, a first approach defines the run of infinite iterations, this method proved to converge to optimal solutions [27,28]. Also, the arbitrary definition of a finite number of iteration as a termination criteria [29], and the use of a predefined execution time [30] are common approaches. However, concerning hybrid approaches, the analysis of dynamic data such as the fitness value has been a modern strategy. In this context, the main idea is the termination of the optimization process when no major difference in values are reached [31]. This line of strategy has been used by authors in [32], where the process of analysis is based in an environment-based method in order to assist in the convergence in an Ant Colony System. Regarding learning-based termination criteria, to

the best of our knowledge, scarce methods have been proposed over the years. Jain et al. in 2001, presented a total of eight methods, which concerns direct, derived and cluster-based termination criteria [33]. The method denominated ClusTerm, manages the individuals in the search space after a few generations when stagnation in the performance was detected. This termination criteria was carried out if certain changes are detected in the elitist clusters generated, this change was measured by a given threshold ϵ. Also, in [34], authors proposed an approach based in landscape analysis coupled with k-means. In order to define a termination criteria, the approach takes advantage of the analysis of fitness through the means of the speed and acceleration of calculated centroids which were related to the convergence in the search process.

Nevertheless, after a detailed literature revision, clustering-based methods seems to be one of the most employed learning-based means to carry out a certain degree of management over the population and termination criteria. Also, most approaches propose classic off-line process and to date no one addressed these two ideas in an unified approach. In this context, we propose a supervised-based architecture capable of balancing the population and the termination criteria, contributing with new lines of investigation to the field. We highlight the promising results obtained, proving the feasibility achieved by our proposed LBMB.

3. Proposed hybrid approach

In this section we illustrate a detailed description concerning LBMB. We present and discuss the main ideas behind the design of the proposed approach. Firstly, in the next subsection we present a general description of the proposition in this work. In Section 4.2, we describe in detail the main components and techniques. Section 4.3, describe the general methodology and work behind LBMB. Finally, Section 4.4, illustrates the proposed algorithm of LBMB.

3.1. General description

The proposed architecture works under a population-based solving strategy; thus, an iterative process is carried out where a population of solutions improves through the search. In this context, this process begins with the generation of a certain number of agents, they interact with the solution space and between them in order to evolve and improve themselves, and this process is repeated until a termination criteria is met. In the literature, parameters such as the termination criteria and population size are usually proposed by authors, however, it is well-known that parameter values can be a sensitive issue for each different problem tackled and even for each instance of the given problem. Thus, the proposed LBMB finds it is core in the adaptabilities given by dynamic parameters values on run-time. For instance, regarding the termination criteria, LBMB will be auto-controlling his working progress through the search. The decision behind the selected value will be considering the worth and possible future profit on keeping iterating the search process. The strong points of the proposal can be described as follows. Firstly, LBMB is focused on the on-line tuning of the population size and termination criteria. The proposed approach focuses on addressing two of the main issues in population-based approaches, the adaptiveness can bring a big profit from the dynamic data generated, such as an early termination in the search process when no possible better solution will be achieved in the current run.

The learning strategy is based on periodic feedback, this constant supply of knowledge helps the architecture to make quick decisions on run-time. In this regard, the proposed approach employs four parameters as threshold to perform different key tasks in the search: α, β, γ, and ϵ. The first task concerns the modification of the population size ruled by α, thus, each time an α number of iterations are performed, the architecture will be changing to a different size. The second task concerns the modification of the termination criteria ruled by β, each time an β number of iterations are performed, the architecture will be modifying the maximum iteration to run the search. The third, concerns the instance on which the regressions are carried out, ruled by γ. Thus, each γ number of iterations are carried out, the model will be processing all the dynamic data, such as feasible solutions, best solutions achieved, infeasible solutions, and the scheme employed scheme to compute their respective percentage values to be employed as independent variables for the regressions to be carried out. Lastly, ϵ defines what we address as iterations of freedom, which comprehends a defined number of iterations at the beginning of the search where the termination criteria will not be modified. Thus, after this ϵ number of iterations is met, LBMB will be able to modify the termination based on self-performance. In Fig. 1, we illustrate an example where α has a value of 250 iterations, β is 250, γ is 1000, and ϵ is to be applied on the first 4000 iterations. On the other hand, regarding the mechanism on which the values of population size and termination criteria are modified, they are defined by schemes. In this work, we defined 4 schemes concerning the population, thus, as illustrated in Table 1, 4 different population sizes can be selected to perform by LBMB. Also, regarding termination criteria, as illustrated in Table 2, 5 different amounts of iterations were added as scheme. Thus, each time a threshold is met, a new scheme will be selected to perform with and new knowledge can be generated. In this regard, the selection mechanism is ruled by a Monte Carlo probabilistic roulette which is influenced by the knowledge generated. The steps proposed for LBMB are described as follows:

1. Set initial parameters.
    - 1.1 for the population-based algorithm.
    - 1.2 for the learning-model: population size.
    - 1.3 for the learning-model: termination criteria.
2. Generate initial population to perform.
3. While the termination criteria is not met.
    - 3.1 diversification operators are carried out.
    - 3.2 the dynamic data generated is sorted.
    - 3.3 intensification operations are carried out.
    - 3.4 the dynamic data generated is sorted.
    - 3.5 check if threshold ϵ is met.
        - 3.5.1 management of dynamic data generated in the current iteration
        - 3.5.2 we check if threshold β is met.
            - 3.5.2.1 the selection mechanism decides which termination criteria scheme will be performing.
            - 3.5.2.2 the selected amount of iteration is added to the current iteration as the new limit to perform.
    - 3.6 check if threshold α is met.
        - 3.6.1 the selection mechanism decides which population size scheme will be performing for the next period of iterations.
        - 3.6.2 the current population size is balanced based in the selection mechanism.

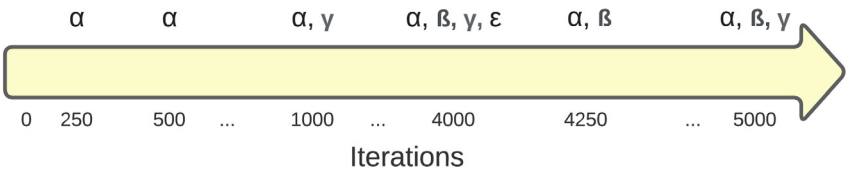


Fig. 1. Graphic illustration of the applied thresholds through the search.

Table 1
Set of amount of agents for LBMB in a 4-way design scheme for the population.

4-way scheme population	
ID	Amount of agents
PI-Grade 1	20
PI-Grade 2	30
PI-Grade 3	40
PI-Grade 4	50

Table 2
Set of iteration values for LBMB in a 5-way design scheme for the termination criteria.

5-way scheme termination criteria	
ID	Amount of iterations
TC-1	50
TC-2	100
TC-3	150
TC-4	200
TC-5	250

3.7 check if threshold γ is met.

3.7.1 carry out the regression analysis for population and termination criteria.

3.7.2 update the selection probabilities based in the output of the performed analysis.

3.2. Methodology

In this subsection, we illustrate practical examples and descriptions of the work performed by LBMB. In this regard, in this first attempt designing this architecture, a 4-way scheme is presented for the management of population, and a 5-way scheme for the termination criteria. Thus, the selection mechanism for the described scheme is ruled by probabilities. The initial values are defined as illustrated in Tables 3 and 4. These values will be modified based on the generated performance knowledge. For instance, if scheme PI-Grade 2 and TC-3 are the minimum value achieved in the regression, the probabilities will be updated as illustrated in Tables 5 and 6.

Regarding the values employed for parameters α, β, γ, and ϵ, partial tests were carried out and defined the first set of values for these parameters in this first version of LBMB. In this context, ϵ concerns a single use threshold defined in 4000 iterations, thus, from iteration 0 to 4000 LBMB will not perform any termination criteria task (module 4). Regarding α, it was set in 250 iterations, so every α amount of iteration a new population size will be sorted to perform. Also, concerning β, it was set in 250, in case this threshold was able to be met, a new scheme will be selected and added to the current iteration. For instance, if we perform this task at iteration 4250 and TC-5 is the selected scheme, the new maximum iteration to perform for LBMB will be set in 4500. Lastly, concerning γ, it was set in 1000, thus, each time this threshold is met, the model will be carrying out regressions for each scheme and the model will be updated through this output.

Table 3
Initial probabilities for population schemes.

4-way scheme population	
ID	Probability
PI-Grade 1	0.25
PI-Grade 2	0.25
PI-Grade 3	0.25
PI-Grade 4	0.25

Table 4
Initial probabilities for termination criteria schemes.

5-way scheme termination criteria	
ID	Probability
TC-1	0.20
TC-2	0.20
TC-3	0.20
TC-4	0.20
TC-5	0.20

Table 5
Updated probabilities for population schemes with PI-Grade 2 as winner.

4-way scheme population	
ID	Probability
PI-Grade 1	0.15
PI-Grade 2	0.55
PI-Grade 3	0.15
PI-Grade 4	0.15

Table 6
Updated probabilities for termination criteria schemes with TC-3 as winner.

5-way scheme termination criteria	
ID	Probability
TC-1	0.15
TC-2	0.15
TC-3	0.40
TC-4	0.15
TC-5	0.15

3.3. LBMB components

In this subsection, we present a detailed explanation and definition of each component proposed for the LBMB architecture.

3.3.1. Module 1: Movements

LBMB follows a population-based algorithm strategy, in which several agents search the solution space of a given problem. In this work, two different approaches are tested, LBMB-SHO and LBMB-CSA. Regarding LBMB-SHO, the search is carried out by the Spotted Hyena Optimizer (SHO) [9] algorithm, where 3 diversification movement operators are employed: Encircling prey, Hunting, and Search for prey. Also, 1 intensification movement operator is employed: Attacking prey. SHO comes from a special type of hyena, the main features comes from the capabilities to group and hunt as a cluster, and the mathematical model is

described as follows. Firstly, the movement encircling the prey is employed and it is mathematically modeled as follows.

illustrated in Eq. (1), where D_h is illustrated in Eq. (2) and includes the distance between the current spotted hyena (P) and the prey (P_p). Also, B and E are coefficients which are defined as illustrated in Eqs. (3) and (4).

$$D_h = \left|B \cdot P_p(x) - P(x)\right| \tag{1}$$

$$P(x+1) = P_p(x) - E \cdot D_h \tag{2}$$

where x represent the current iteration, B and E are coefficient, P_p depicts the prey, P is the current spotted hyena, thus, D_h correspond to the distance between the current spotted hyena and the prey.

$$B = 2 \cdot rd_1 \tag{3}$$

$$E = 2h \cdot rd_2 - h \tag{4}$$

$$h = 5 - (x * (5/X)) \tag{5}$$

where $x = 1, 2, 3, \ldots, X$, rd_1 and rd_2 are random values in [0,1]. Also, h was defined by authors as the parameter managing the diversification and intensification, the values linearly decrease from 5 to 0 through the search. On the other hand, the second movement operator is called Hunting, and is mathematically modeled as illustrated in Eqs. (6), (7), and (8). Here, P_k comprehends the position for each spotted hyena in the population and P_h denotes the best spotted hyena in the population. Also, N is the number of spotted hyenas.

$$D_h = |B \cdot P_h - P_k| \tag{6}$$

$$P_k = P_h - E \cdot D_h \tag{7}$$

$$C_h = P_k + P_{k+1} + \cdots + P_{k+N} \tag{8}$$

The third movement operator is attacking Prey, on which the proposed equation is illustrated in (9). Here, the position of each agent in the population is updated. Lastly, SHO employs the movement search prey, which focuses on passive exploration. The work is done by coefficients E and B, which employs random values to force the search agents to move far away from the prey. This mechanism allows the algorithm to search globally.

$$P(x+1) = C_h/N \tag{9}$$

where $P(x+1)$ observes the best solution and updates the positions of other search agents influenced by the position of the best agent, the spotted hyena attacking the prey constantly updates their position.

Regarding LBMB-CSA, the search is carried out by CSA [10], where multiple features can be presented, such as the crows living in the form of flock, memorize the personal hidden location, sometimes they follow each other to do thievery, and give protection to their respective caches by a probability. The initial definitions are as follows: N corresponds to the number of crows (flock size), also, each crow has a memory which is illustrated as $m^{i,x}$. This represents the best position that $crow^{i,x}$ has obtained to that point in the search (x current iteration). On the other hand, CSA states that there will be a $crow^{j,x}$ with the objective of visiting $m^{i,x}$. Thus, in the same iteration that $crow^{j,x}$ makes his move, the $crow^{i,x}$ will follow up, and two scenarios may happen and are described as follows.

- scenario 1: $crow^{j,x}$ never notice the presence of $crow^{i,x}$, which is represented as $r_j \geq AP^{j,x}$. Here, r_j is a random number between 0 and 1 and $AP^{j,x}$ denotes the awareness probability of $crow^{j,x}$. This AP defines the ratio balance on which basic CSA performs diversification (larger values for AP) and intensification (smaller values for AP). Thus, in this case, $crow^{i,x}$ will get the hidden place $m^{j,x}$ and the new position of $crow^{i,x}$ will be updated as follows.

$$crow^{i,x+1} = crow^{i,x} + r_i \times fl^{i,x} \times (m^{j,x} - crow^{i,x}) \tag{10}$$

Here, r_i depicts a random number with random distribution between 0 and 1, and $fl^{i,x}$ is the flight length of $crow^{i,x}$, where small values leads to local search ($fl < 1$) and larger values lead to global search ($fl > 1$).

- scenario 2: $crow^{j,x}$ notice the presence of $crow^{i,x}$, in this case, $crow^{j,x}$ will lead $crow^{i,x}$ to a random position in the search space.

3.3.2. Module 2: Administrator

The objective of this module comprehends the management of the population size, termination criteria, and the selection mechanism of the scheme. In this context, firstly, the module can add randomly generated or remove agents from the population, this process is carried out after a new scheme is selected. Also, concerning the termination criteria, the module can determine a possible better amount of iteration to keep working before terminating the search in order to avoid convergence stagnation in the solutions. Concerning the selection of the scheme, they are the defined configuration sizes that LBMB will employ through the search. The selection process follows a roulette system, where the items to be placed and selected works under a stochastic-simulation mechanism.

The probabilities of each scheme to be selected will be modified based on the predictive analysis carried out for each scheme in each learning module, giving a bigger priority to the possible best to perform and reach better results.

3.3.3. Module 3: Learning-population

This module comprehends two major objectives, the regression analysis associated with the prediction of better fitness values under a suitable amount of agents, and the management of information needed to carry out the analysis. Firstly, the regression analysis is carried out by the means of multiple linear regression, where the fitted function is of the form.

$$y = wx_j + \cdots + wx_n + b \quad \text{and} \quad j = \{1, 2, \ldots, n\} \tag{11}$$

where, y corresponds to the dependent variable, which is the fitness and the value objective to predict. Concerning x_i to x_n, represent the independent variables, which correspond to three optimization metrics and the employed scheme. The metrics studied in this work are the percentage of feasible solutions generated (x_{f-i}), percentage of infeasible solutions generated (x_{inf-i}), and best solution achieved (x_{best-i}) with $i = \{1, 2, 3, 4\}$. In this module, we present the close relationship between the fitness convergence with the balance of population and the performance achieved in a finite number of iterations. Regarding our proposed learning-model, we define four fitted functions for the proposed population schemes, they are represented as follows.

$$y_{PI-Grade1} = w_i x_{f-1} + w_i x_{inf-1} + w_i x_{best-1} + w_i x_{PI-1} + b_i \tag{12}$$

$$y_{PI-Grade2} = w_i x_{f-2} + w_i x_{inf-2} + w_i x_{best-2} + w_i x_{PI-2} + b_i \tag{13}$$

$$y_{PI-Grade3} = w_i x_{f-3} + w_i x_{inf-3} + w_i x_{best-3} + w_i x_{PI-3} + b_i \tag{14}$$

$$y_{PI-Grade4} = w_i x_{f-4} + w_i x_{inf-4} + w_i x_{best-4} + w_i x_{PI-4} + b_i \tag{15}$$

Thus, LBMB will be making decisions based on the results of these functions. Moreover, as the benchmark solved are minimization problems, the defined function to influence the probability for each scheme to be selected can be illustrated as follows.

$$f(O_{scheme}) = \text{MIN}(y_{PI-Grade1}, y_{PI-Grade2}, y_{PI-Grade3}, y_{PI-Grade4}) \quad (16)$$

where O_{scheme} represents the best scheme option to perform for an α amount of iteration in order to have higher possibilities to achieve better results.

3.3.4. Module 4: Learning-Termination criteria

The main objective comprehends the regression analysis associated with the measurement of performance and profit on iterating for a certain amount of future iteration. The aim is to take into consideration the maximization of the output achieved (fitness) employing the lesser possible processing time and resources. The regression analysis is carried out by the means of multiple linear regression, where the fitted function is of the form.

$$y = wx_j + \cdots + wx_n + b \quad \text{and} \quad j = \{1, 2, \ldots, n\} \quad (17)$$

where, y correspond to the dependent variable, which is the fitness and the value objective we want to predict. Concerning x_i to x_n, represent the independent variables, which correspond to three optimization metrics and the corresponding scheme. The metrics studied in this works are the percentage of feasible solutions generated (x_{f-i}), percentage of infeasible solutions generated (x_{inf-i}), and best solution achieved while performing each respective scheme (x_{TC-i}) with $i = \{1, 2, 3, 4, 5\}$. Regarding our proposed learning-model, we define five fitted functions for the proposed termination criteria scheme, they are represented as follows.

$$y_{TC-1} = w_i x_{f-1} + w_i x_{inf-1} + w_i x_{best-1} + w_i x_{TC-1} + b_i \quad (18)$$

$$y_{TC-2} = w_i x_{f-2} + w_i x_{inf-2} + w_i x_{best-2} + w_i x_{TC-2} + b_i \quad (19)$$

$$y_{TC-3} = w_i x_{f-3} + w_i x_{inf-3} + w_i x_{best-3} + w_i x_{TC-3} + b_i \quad (20)$$

$$y_{TC-4} = w_i x_{f-4} + w_i x_{inf-4} + w_i x_{best-4} + w_i x_{TC-4} + b_i \quad (21)$$

$$y_{TC-5} = w_i x_{f-5} + w_i x_{inf-5} + w_i x_{best-5} + w_i x_{TC-5} + b_i \quad (22)$$

Thus, the decision making is based on the results of these functions. Moreover, as the benchmark tackled correspond to minimization problems, the defined function to influence the probability for each scheme to be selected was defined as follows.

$$f(O_{scheme}) = MIN(y_{PI-Grade1}, y_{PI-Grade2}, y_{PI-Grade3}, y_{PI-Grade4}) \quad (23)$$

where O_{scheme} represents the best scheme option to perform for the next β amount of iteration in order to have higher possibilities to keep converging.

3.4. Proposed algorithm

In this subsection, we illustrate the designed algorithm proposed. In Algorithm 1 we present the work performed in each module explained in Section 3.3.

Algorithm 1 *Proposed LBMB*

Module 1 (Movements): set initial parameters required for movement operators
Module 3 (Learning-Population): set parameters and data structures
Module 4 (Learning-Termination criteria): set parameters and data structures
Module 1 (Movements): generate initial population
while ($i \leq$*MaximumIteration*) **do**
  Module 1 (Movements): perform intensification family of operators
  Module 1 (Movements): perform diversification family of operators
  Module 3 (Learning-Population): Management of dynamic data generated
  if threshold ϵ is met **then**
    Module 4 (Learning-Termination criteria): Management of dynamic data generated
    if threshold β is met **then**
      Module 2 (Administrator): Select new termination criteria scheme
      Module 2 (Administrator): Update new MaximumIteration
    end if
  end if
  if threshold α is met **then**
    Module 2 (Administrator): Select new population scheme
    Module 2 (Administrator): Balance the population
  end if
  if threshold γ is met **then**
    Module 3 (Learning-Population): Regression analysis
    Module 4 (Learning-Termination criteria): Regression analysis
    Module 2 (Administrator): Update probabilities for Population schemes
    Module 2 (Administrator): Update probabilities for termination criteria schemes
  end if
end while

4. Experimental results

This section presents the design behind the experimentation performed over LBMB, results achieved, comparison against State-of-the-art algorithms, general ideas and discussion in the performance observed.

4.1. Benchmark test functions

In this work, we make use of two sets of well-known standard benchmark functions, in order to test the performance. The first set consist on fifteen functions are divided into three main categories, such as unimodal [35] illustrated in Eqs. (24)–(27), multimodal [36] illustrated in Eqs. (28)–(32), and multimodal with fixed-dimension [35,36] illustrated in Eqs. (33)–(38). These functions include multiple features which justify their usage in this work to be solved by the proposed approach. For instance, f_1 to f_5 have one global optimum and has no local optima, which makes this first group of functions suitable to measure the convergence rate and intensification capacity of the architecture. The second and third group of functions, f_4 to f_9 and f_{10} to f_{15} respectively, they have multiple local solutions besides the global optimum. Thus, they are useful measuring how efficient LBMB will be avoiding local optima and his diversification abilities. Also, it is well-known that multimodal functions, f_5 to f_9, corresponds to a group of very difficult problem to solve for optimization algorithms, where there is an exponentially increase in number of dimensions [37]. Regarding the second set illustrated in Eq. (39)–(47), they are high dimension functions proposed in [38], the main objective designing this benchmark is to test the reliability and robustness of the optimization approach [39]. This issue can be measured by the employment of points further away from the optimum values displaying important differences in reliability and robustness represented by the amount of function evaluations or computational time.

Unimodal functions:

Sphere Function

$$f_1(x) = f(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} x_i^2 \tag{24}$$

Schwefel's Function No. 2.22

$$f_2(x) = \sum_{i=1}^{n} |x_i| + \prod_{i=1}^{n} |x_i| \tag{25}$$

Schwefel's Function No. 1.2

$$f_3(x) = \sum_{i=1}^{n} \left(\sum_{j=1}^{i} x_j \right)^2 \tag{26}$$

Generalized Rosenbrock's Function

$$f_4(x) = \sum_{i=1}^{n-1} (100(x_i^2 - x_{i+1})^2 + (1 - x_i)^2) \tag{27}$$

Multimodal functions:

Generalized Schwefel's Function No. 2.26

$$f_5(x) = -\sum_{i=1}^{n} x_i sin(\sqrt{|x_i|}) \tag{28}$$

Generalized Rastrigin's Function

$$f_6(x) = 10n + \sum_{i=1}^{n} (x_i^2 - 10cos(2\pi x_i)) \tag{29}$$

Ackley's Function

$$f_7(x) = -20exp(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^{n} x_i^2}) - exp(\frac{1}{n} \sum_{i=1}^{n} cos(2\pi x_i)) + 20 + exp(1) \tag{30}$$

Generalized Griewank's Function

$$f_8(x) = 1 + \sum_{i=1}^{n} \frac{x_i^2}{4000} - \prod_{i=1}^{n} cos(\frac{x_i}{\sqrt{i}}) \tag{31}$$

Generalized Penalized Function

$$f_9(x) = \frac{\pi}{n} \times \left\{ 10 \sin^2(\pi y_1) + \sum_{i=1}^{n-1} (y_i - 1)^2 \left[1 + 10 \sin^2(\pi y_{i+1}) \right] + (y_n - 1)^2 \right\} + \sum_{i=1}^{n} u(x_i, 10, 100, 4) \tag{32}$$

where $u(x_i, a, k, m)$ is equal to

1. $k(x_i - a)^m$ if $x_i > a$
2. 0 if $-a \leq x_i \leq a$
3. $k(-x_i - a)^m$ if $x_i < -a$

and

1. $y_i = 1 + \frac{1}{4}(x_i + 1)$

Table 7
Values of a_{ij}, c_i, and p_{ij} for function $f_14(x)$; $n = 3$ and $j = 1, 2, 3$.

i	a_{ij}			c_i	p_{ij}		
1	3	10	30	1	0.3689	0.1170	0.2673
2	0.1	10	35	1.2	0.4699	0.4387	0.7470
3	3	10	30	3	0.1091	0.8732	0.5547
4	0.1	10	30	3.2	0.03815	0.5743	0.8828

Multimodal functions with fixed dimensions:

Shekel's Foxholes Function

$$f_{10}(x) = \left[\frac{1}{500} + \sum_{j=1}^{25} \frac{1}{j + \sum_{i=1}^{2} (x_i - a_{i,j})^6} \right]^{-1} \tag{33}$$

where:

$$a_{i,j} = \begin{bmatrix} -32 & -16 & 0 & 16 & 32 & -32 & \ldots & 0 & 16 & 32 \\ -32 & -32 & -32 & -32 & -32 & -16 & \ldots & 32 & 32 & 32 \end{bmatrix}$$

Six-hump Camel Back Function

$$f_{11}(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 4x_2^4 \tag{34}$$

Branin's Function

$$f_{12}(x) = \left(x_2 - \frac{5.1x_1^2}{4\pi^2} + \frac{5x_1}{\pi} - 6 \right)^2 + 10 \left(1 - \frac{1}{8\pi} \right) \cos(x_1) + 10 \tag{35}$$

Goldstein–Price Function

$$f_{13}(x) = \left[1 + (x_1 + x_2 + 1)^2 \times \left(19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2 \right) \right] \times \left[30 + (2x_1 - 3x_2)^2 \left(18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2 \right) \right] \tag{36}$$

Hartman's Function No. 1

$$f_{14}(x) = -\sum_{i=1}^{4} c_i \, e \left[-\sum_{j=1}^{3} a_{i,j} \left(x_j - p_{i,j} \right)^2 \right] \tag{37}$$

where the values of a, c and p are tabulated in Table 7.

Hartman's Function No. 2

$$f_{15}(x) = -\sum_{i=1}^{4} c_i \, e \left[-\sum_{j=1}^{6} a_{i,j} \left(x_j - p_{i,j} \right)^2 \right] \tag{38}$$

where the values of a, c and p are tabulated in Table 8.

High-dimension functions:

Extended Rosenbrock Function — f_{16}

$$f_{2i-1}(x) = 10(x_{2i} - x_{i-1}^2) \quad f_{2i}(x) = 1 - x_{2i-1} \tag{39}$$

Extended Powell Singular Function — f_{17}

$$f_{4i-3}(x) = x_{4i-3} + 10x_{4i-2} \quad f_{4i-2}(x) = 5^{1/2}(x_{4i-1} - x_{4i}) \quad f_{4i-1}(x) = (x_{4i-2} - x_{4i-1})^2 \quad f_{4i}(x) = 5^{1/2}(x_{4i-3} - x_{4i})^2 \tag{40}$$

Penalty I Function — f_{18}

$$f(x) = \sum_{i=1}^{n} (a^{1/2}(x_{i-1})) + \sum_{j=1}^{n} (x_j^2) - \frac{1}{4} \tag{41}$$

Table 8
Values of a_{ij}, c_i, and p_{ij} for function $f_15(x)$; $n = 6$ and $j = 1, 2, \ldots, 6$.

i	a_{ij}						c_i	p_{ij}					
1	10	3	17	3.5	1.7	8	1	0.131	0.169	0.556	0.012	0.828	0.588
2	0.05	10	17	0.1	8	14	1.2	0.232	0.413	0.830	0.373	0.100	0.999
3	3	3.5	1.7	10	17	8	3	0.234	0.141	0.352	0.288	0.304	0.665
4	17	8	0.05	10	0.1	14	3.2	0.404	0.882	0.873	0.574	0.109	0.038

Variably Dimensioned Function — f_{19}

$$f(x) = \sum_{i=1}^{n}(x_i - 1) + \sum_{i=1}^{n} i(x_i - 1) + \sum_{i=1}^{n}(i(x_i - 1))^2 \tag{42}$$

Linear-full Rank Function — f_{20}

$$\begin{aligned} f_i(x) &= x_i - \frac{2}{m}(\sum_{j=1}^{n} x_j) - 1,\ 1 \le i \le n \\ f_i(x) &= -\frac{2}{m}(\sum_{j=1}^{n} x_j) - 1,\ n < i \le m \end{aligned} \tag{43}$$

Discrete Integral Equation — f_{21}

$$\begin{aligned} f_i(x) = x_i + h[(1 - t_i)\sum_{j=1}^{n} t_j(x_j + t_j + 1)^3 \\ + t_j \sum_{j=1}^{n}(1 - t_i)(x_j + t_j + 1)^3]/2 \\ h = 1/(n+1),\ t_i = ih,\ x_0 = x_{n+1} = 0 \end{aligned} \tag{44}$$

Brown Almost-linear Function — f_{22}

$$\begin{aligned} f_i(x) &= x_i + \sum_{j=1}^{n} x_j - (n+1),\ 1 \le i < n \\ f_n(x) &= (\prod_{j=1}^{n} x_j) - 1 \end{aligned} \tag{45}$$

Discrete Boundary Value Function — f_{23}

$$\begin{aligned} f_i(x) = 2x_i - x_{i-1} - x_{i+1} + h^2(x_i + t_i + 1)^3/2 \\ h = 1/(n+1),\ t_i = ih,\ x_0 = x_{n+1} = 0 \end{aligned} \tag{46}$$

Trigonometric Function — f_{24}

$$f_i(x) = n - \sum_{j=1}^{n} \cos x_j + i(1 - \cos x_i) - \sin x_i \tag{47}$$

4.2. Multidimensional knapsack problem

The Multidimensional Knapsack Problem (MKP) can be defined as a NP-hard problem and can be considered as the generalized form of the classic Knapsack Problem (KP). The main objective of MKP is to search for a subset of given objects that maximize the total profit while satisfying all constraints on resources. Also, the KP is a well-known optimization problem which has been applied in multiple real-world fields, such as cryptography, allocation problems, scheduling, and production [40,41]. The model can be stated as follows.

$$\text{Maximize} \quad \sum_{j=1}^{n} c_j x_j$$

$$\text{Subject to} \quad \sum_{j=1}^{n} a_{ij} x_j \le b_i, \quad i \in M = 1, 2, \ldots, m$$

$$x_j \in \{0, 1\}, \quad j \in N = 1, 2, \ldots, n$$

Table 9
Configuration details from MKP instances employed in this work.

ID	Test Problem	Optimal solution	n	m
mknapcb1	5.100.00	24 381	100	5
	5.100.01	24 274	100	5
	5.100.02	23 551	100	5
	5.100.03	23 534	100	5
	5.100.04	23 991	100	5
mknapcb2	5.250.00	59 312	250	5
	5.250.01	61 472	250	5
	5.250.02	62 130	250	5
	5.250.03	59 463	250	5
	5.250.04	58 951	250	5
mknapcb3	5.500.00	120 148	500	5
	5.500.01	117 879	500	5
	5.500.02	121 131	500	5
	5.500.03	120 804	500	5
	5.500.04	122 319	500	5
mknapcb4	10.100.00	23 064	100	10
	10.100.01	22 801	100	10
	10.100.02	22 131	100	10
	10.100.03	22 772	100	10
	10.100.04	22 751	100	10
mknapcb5	10.250.00	59 187	250	10
	10.250.01	58 781	250	10
	10.250.02	58 097	250	10
	10.250.03	61 000	250	10
	10.250.04	58 092	250	10
mknapcb6	10.500.00	117821	500	10
	10.500.01	119 249	500	10
	10.500.02	119 215	500	10
	10.500.03	118 829	500	10
	10.500.04	116 530	500	10

where n is the number of items and m is the number of knapsack constraints with capacities b_i. Each item j requires a_{ij} units of resource consumption in the ith knapsack and yields c_j units of profit upon inclusion. The goal is to find a subset of items that yields maximum profit without exceeding the resource capacities. Additionally, SHO was initially designed to work on a continuous space, in order to tackle the MCDP, SCP, and MKP, a transformation of domain is needed. In this work, this task is performed by applying binarization strategies, where each strategy is composed by a transfer function [42] and a discretization method. In this regard, we follow the strategy proposed in [43], which employs the transfer function V_4 + Elitist discretization.

In order to test adaptability and performance of LBMB tackling discrete problems, the MKP was selected to be solved. In this regard, 6 different instance sets from the Beasleys ORlibrary were employed. The details concerning the solved benchmark are illustrated in Table 9.

4.3. Experimental setup

In order to present a detailed experimental process over LBMB-SHO and LBMB-CSA, we designed a testing phase comprehending a four step process. In this context, the first step concerns the comparison of results achieved by our architecture against state-of-the-art methods reported in [37,38,44–46]. For the first set we include classic and highly employed algorithms, such as

Table 10
Configuration parameters for LBMB-SHO, LBMB-CSA, SHO, CSA, CSA+IRace, and SHO+IRace.

	Parameters	Values
LBMB-SHO	Search agents	Scheme(20, 30, 40, 50)
	Control parameter (h)	[5, 0]
	M constant	[0.5, 1]
	Number of generations	initial(10 000), Scheme(50, 100, 150, 200, 250)
	α	250
	β	250
	γ	1000
	ϵ	4000
SHO	Search agents	30
	Control parameter (h)	[5, 0]
	M constant	[0.5, 1]
	Number of generations	10 000
SHO+IRace	Search agents	45
	Control parameter (h)	[5, 0]
	M constant	[0.5, 1]
	Number of generations	6500
LBMB-CSA	Search agents	Scheme(20, 30, 40, 50)
	r_i	[0, 1]
	r_j	[0, 1]
	Awareness probability (AP)	0.1
	Flight length (fl)	2
	Number of generations	initial(10 000), Scheme(50, 100, 150, 200, 250)
	α	250
	β	250
	γ	1000
	ϵ	4000
CSA	Search agents	50
	r_i	[0, 1]
	r_j	[0, 1]
	Awareness probability (AP)	0.1
	Flight length (fl)	2
	Number of generations	10 000
CSA+IRace	Search agents	89
	r_i	[0, 1]
	r_j	[0, 1]
	Awareness probability (AP)	0.1
	Flight length (fl)	2
	Number of generations	9140

Particle Swarm Optimization (PSO) [47] and Differential Evolution (DE) [48]. Also, modern approaches such as Gravitational Search Algorithm (GSA) [49], Whale Optimization Algorithm (WOA) [50], Vapor–Liquid Equilibrium (VLE) [51], and an hybrid between Nelder–Mead Algorithm and Dragonfly Algorithm (INMDA) [52]. For the second set, Standard Nelder–Mead Simplex (SNMS) and Adaptive Nelder–Mead Simplex (ANMS) [38], and a Nelder–Mead with perturbed centroid (PNM) [46]. Regarding the second step, we compare against a well-known method specially designed to tackle the tuning of parameters, we implemented an IRace [53] approach based on traditional SHO and CSA. Concerning the third step, we take a closer look to the performance achieved to traditional SHO and CSA against the corresponding LBMB-SHO and LBMB-CSA. Lastly, in the four step we compare the performance achieved solving the MKP between LBMB-SHO, LBMB-CSA, CHO and CSA assisted by IRace, and classic implementation of SHO and CSA. The main objective behind this last step, is to test the performance of the proposed architecture solving discrete optimization problems. The main idea is for LBMB to have the capabilities to tackle diverse/multiple optimization problem with an unique set of defined parameters. Within the second, third, and four steps, in order to further analyze and demonstrate the improvements in the performance, Wilcoxon's signed rank (Mann and Whitney 1947), a statistical analysis test, was performed to support the results. Additionally, in order to carry out the statistical analysis of this phase, we make use of the RStudio software to conduct the test.

Regarding the details about the implementations, they are illustrated in Table 10. Also, the implementation was done in python 3.7 and ran in a windows 10 environment with 64 bits on Core i-5 processor with 2.40 GHz and 8 GB memory. Finally, regarding the experimentation phase, for each benchmark function the algorithm utilizes 30 independent runs.

4.4. Performance comparison

In this subsection we illustrate in detail the results, analysis, comparison against reported state-of-the-art algorithms. Also, we highlight improvements, achievements, and problems observed in the performance.

4.4.1. First experimentation step

Firstly, all results achieved by our proposed approach were rounded to four decimals. The results illustrated concerning the state-of-the-art methods employed were rounded in Tables 11, 12, 13, and 14 to four decimals, using scientific notation, only for presentation purposes. Nevertheless, all computations were carried out using the reported decimals by their respective authors.

Regarding the performance on unimodal functions, Table 11 illustrates the comparison tackling functions f_1 to f_4, the Average (Avg) and Standard deviation (StdDev) are presented and compared, and we highlight in bold the best values reached. It is clear and surprising how well performed LBMB-SHO and LBMB-CSA, they were efficient algorithms tackling the two sets of benchmark functions. Moreover, in the first set, the smallest values reached corresponding to the StdDev shows that it is a very solid algorithm. Also, it is well-known that unimodal functions can help us to measure the exploitation capabilities of our proposed approach, thus giving us a positive first impression concerning this issue. Concerning the multimodal functions and

Table 11
Results comparison in unimodal benchmark functions.

F	LBMB-CSA		LBMB-SHO		WOA		DE		GSA		PSO		VLE		INMDA	
	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev
f_1	**0.0000**	**0.0000**	**0.0000**	**0.0000**	1.4100×10^{-30}	4.9100×10^{-30}	8.2000×10^{-14}	5.9000×10^{-14}	2.5300×10^{-16}	9.6700×10^{-17}	1.3600×10^{-4}	2.0200×10^{-4}	4.4989×10^{-7}	1.413×10^{-6}	**0.0000**	**0.0000**
f_2	1.9634×10^{-8}	2.1923×10^{-8}	**0.0000**	**0.0000**	1.0600×10^{-21}	2.3900×10^{-21}	1.5000×10^{-9}	9.9000×10^{-10}	5.5655×10^{-2}	0.1941	4.2144×10^{-2}	4.5421×10^{-2}	3.0840×10^{-6}	$6.0498 \times 10^{-}6$	**0.0000**	**0.0000**
f_3	2.8536×10^{-12}	6.4462×10^{-12}	**0.0000**	**0.0000**	5.3900×10^{-7}	2.9300×10^{-6}	6.8000×10^{-11}	7.4000×10^{-11}	8.9353×10^{2}	3.1896×10^{2}	70.126	22.119	5.2020	0.7986	**0.0000**	**0.0000**
f_4	3.7487×10^{-14}	5.7753×10^{-14}	5.1260	7.1011	27.866	0.7636	**0.0000**	**0.0000**	67.543	62.225	96.718	60.116	79.199	37.400	**0.0000**	**0.0000**

Table 12
Results comparison in multimodal benchmark functions.

F	LBMB-CSA		LBMB-SHO		WOA		DE		GSA		PSO		VLE		INMDA	
	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev
f_5	$-12\,569.4866$	4.6375×10^{-12}	-5.8222×10^{3}	8.1121	-5.0808×10^{3}	6.9580×10^{2}	-1.1080×10^{4}	5.7470×10^{2}	-2.8211×10^{3}	4.9304×10^{2}	-4.8413×10^{3}	1.1528×10^{3}	-1.2566×10^{4}	68.705	-2245.1500	2.8400
f_6	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**	69.200	38.800	25.968	7.4701	46.704	11.629	34.5830	17.8860	**0.0000**	**0.0000**
f_7	3.2654×10^{-8}	6.7535×10^{-8}	0.1000	1.4107	7.4043	9.8976	9.7000×10^{-8}	4.2000×10^{-8}	6.2087×10^{-2}	0.23628	0.27602	0.50901	3.1704	3.9211	2.9600×10^{-17}	1.6200×10^{-16}
f_8	1.3744×10^{-14}	1.8669×10^{-14}	**0.0000**	**0.0000**	2.8900×10^{-4}	1.5860×10^{-3}	**0.0000**	**0.0000**	27.702	5.0403	9.2150×10^{-3}	7.7240×10^{-3}	0.5074	0.5041	**0.0000**	**0.0000**
f_9	1.8285	5.5733×10^{-16}	16.2587	13.8186	0.3397	0.2149	7.9000×10^{-15}	8.0000×10^{-15}	1.7996	0.95114	6.9170×10^{-3}	2.6301×10^{-2}	0.2369	0.2877	4.700×10^{-32}	1.4900×10^{-31}

Table 13
Results comparison in multimodal benchmark functions with fixed-dimension.

F	LBMB-CSA		LBMB-SHO		WOA		DE		GSA		PSO		VLE		INMDA	
	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev	Avg	StdDev
f_{10}	**1.0000**	**0.0000**	6.3874	5.1618	2.1120	2.4986	0.99800	3.3000×10^{-16}	5.8598	3.8313	3.6272	2.5608	0.99800	2.5294×10^{-7}	N/A	N/A
f_{11}	0.0000	0.0000	2.0290×10^{-17}	2.9159×10^{-17}	−1.0316	4.2000×10^{-7}	**−1.0316**	3.1000×10^{-13}	**−1.0316**	4.8800×10^{-16}	**−1.0316**	6.2500×10^{-16}	−1.0315	1.8408×10^{-4}	N/A	N/A
f_{12}	−1.0235	0.0103	0.4953	0.1777	0.39791	2.7000×10^{-5}	0.39789	9.9000×10^{-9}	0.39789	0.0000	0.39789	0.0000	0.39815	4.5697×10^{-4}	N/A	N/A
f_{13}	32.6845	3.8918×10^{-15}	32.6846	4.0269×10^{-5}	**3.0000**	4.2200×10^{-15}	**3.0000**	2.0000×10^{-15}	**3.0000**	4.1700×10^{-15}	**3.0000**	1.3300×10^{-15}	3.0097	1.6256×10^{-2}	N/A	N/A
f_{14}	−2.0080	0.0000	−2.0080	1.4829×10^{-8}	−3.8562	2.7060×10^{-3}	N/A	N/A	−3.8628	2.2900×10^{-15}	−3.8628	2.5800×10^{-15}	−3.8628	6.6880×10^{-5}	N/A	N/A
f_{15}	−1.1676	1.8577×10^{-16}	−1.1676	2.2514×10^{-9}	−2.9811	0.37665	N/A	N/A	−3.3178	2.3081×10^{-2}	−3.2663	6.0516×10^{-2}	−3.3179	2.1311×10^{-2}	N/A	N/A

Table 14
Results comparison in high dimension benchmark functions.

F	LBMB+SHO	LBMB+CSA	PNM	SNMS	ANMS
	Best	Best	Best	Best	Best
f_{16}	1.0361×10^{-8}	**0**	4.1627×10^{-2}	21.314	1.3705×10^{-9}
f_{17}	**0**	**0**	1.3758×10^{-4}	0.0024	1.9181×10^{-5}
f_{18}	0.0015	0.0015	1.4763×10^{-3}	2.6319×10^{6}	4.3179×10^{-4}
f_{19}	0.0011	2.1839×10^{-14}	3.5724×10^{1}	72.101	1.8116×10^{-8}
f_{20}	1.4774×10^{-11}	**0**	7.4923×10^{-8}	0.0280	2.7243×10^{-8}
f_{21}	0.4602	0.4602	7.9784×10^{-8}	5.1304×10^{-4}	2.2644×10^{-7}
f_{22}	0.0322	1.5123×10^{-6}	6.4715×10^{-6}	N/A	N/A
f_{23}	$1,0672 \times 10^{-6}$	1.0672×10^{-6}	3.0709×10^{-7}	5.3411×10^{-6}	5.3085×10^{-6}
f_{24}	**0**	1.3999×10^{-8}	7.6536×10^{-7}	4.8213×10^{-5}	9.6665×10^{-7}

Table 15
Results comparison of SHO+IRace vs. LBMB-SHO.

F	Opt	SHO + IRace					LBMB-SHO				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
f_1	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	122.7566	**0.0000**	**0.0000**	**0.0000**	**0.0000**	18.6433
f_2	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	145.4655	**0.0000**	**0.0000**	**0.0000**	**0.0000**	33.4892
f_3	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	554.6584	**0.0000**	**0.0000**	**0.0000**	**0.0000**	177.5099
f_4	0	0.1611	62.9045	18.2664	12.0161	156.9801	0.0668	55.6351	5.1260	7.1011	25.0293
f_5	−12,569.487	-1.2771×10^{4}	-1.5055×10^{4}	-0.3426×10^{4}	29.1731	234.6455	-1.2692×10^{4}	15.2518	-5.8222×10^{3}	8.1121	17.6527
f_6	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	285.5517	**0.0000**	**0.0000**	**0.0000**	**0.0000**	20.1004
f_7	0	4.4409×10^{-16}	1.1438	1.2727	4.8821	277.2413	4.4408×10^{-16}	0.9341	0.1000	1.4107	28.6423
f_8	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	299.1335	**0.0000**	**0.0000**	**0.0000**	**0.0000**	22.6069
f_9	0	1.8617	170.0366	41.5703	51.1004	375.4450	2.3498	73.8897	16.2587	13.8186	170.3205
f_{10}	1	1.1782	10.7187	10.1649	3.1969	263.5944	0.0033	11.7187	6.3874	5.1618	27.2279
f_{11}	−1.0316285	9.7163×10^{-20}	2.6591×10^{-17}	1.9508×10^{-17}	2.9369×10^{-17}	151.1100	6.3595×10^{-17}	4.5194	2.0290×10^{-17}	2.9159×10^{-17}	4.0946
f_{12}	0.397887	0.1398	3.2165	13.1491	12.5237	137.3323	0.3589	2.2683	0.4953	0.1777	4.2014
f_{13}	3	32.6845	66.2712	32.6866	0.0033	283.1000	29.6845	55.8719	32.6846	4.0269×10^{-5}	5.3021
f_{14}	−3.86	−2.0081	2.1541	−2.0081	2.5764×10^{-7}	271.1564	−2.0081	1.9282	−2.0080	1.4829×10^{-8}	11.7642
f_{15}	−3.32	−1.1676	2.1575	−1.1676	1.5261×10^{-7}	189.8891	−1.1676	2.1524	−1.1676	2.2514×10^{-9}	15.2136
f_{16}	0	4.5739×10^{-6}	79.7268	36.0699	39.5128	67.8109	1.0361×10^{-8}	75.6147	28.8705	38.1926	21.4518
f_{17}	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	101.6144	**0.0000**	**0.0000**	**0.0000**	**0.0000**	30.0628
f_{18}	unkown	0.0015	4.4121	0.0016	0.0002	167.6020	0.0015	0.0303	0.0016	0.0001	45.5892
f_{19}	0	0.0667	177.0903	76.3744	141.6495	65.6123	0.0011	1.3802×10^{16}	5.5042×10^{15}	1.1008×10^{16}	10.6033
f_{20}	0	8.8776×10^{-8}	83.5577	14.5456	45.9967	74.5253	1.4774×10^{-11}	2.6165	2.0903×10^{-7}	6.0620×10^{-7}	18.4379
f_{21}	0	0.4602	6.2146	1.2449	1.5695	441.0214	0.4602	22.1613	0.8526	1.1771	76.3696
f_{22}	0	0.2440	28 228.8185	5132.4053	10 233.8470	99.7350	0.0322	2.6867×10^{3}	6.9827×10^{3}	1.1401×10^{4}	23.1293
f_{23}	0	1.0672×10^{-6}	0.2359	1.0692×10^{-6}	3.4011×10^{-9}	95.5370	1.0672×10^{-6}	0.0001	1.0679×10^{-6}	2.2910×10^{-9}	16.2799
f_{24}	0	**0.0000**	7.6029	0.1156	0.4065	111.6935	**0.0000**	19.4323	0.0012	0.0037	31.2105

multimodal functions with fixed dimensions, both groups help us to measure the ability of our proposed approaches to explore the search space. Tables 12 and 13 illustrate the results and comparison on functions f_5 to f_9 and f_{10} to f_{15} correspondingly. The Average (Avg) and Standard deviation (StdDev) are presented, and we highlight in bold the best values reached. In this regard, our proposed architecture got promising results and small Avg and StdDev values once again, proving to be a competitive approach able to tackle optimization problems. Regarding the second set, the comparison of functions f_{16} to f_{24} is presented in Table 14, where the authors only report the best (Best) values reached. In this regard, it is well-known the difficulties implicated in higher dimensional problems where the robustness is tested to it is limits. The proposed approaches based in LBMB presents good results in comparison to the other 3 methods, thus, the next step would be testing the proposed architecture on harder optimization problems related to the industry as future work.

4.4.2. Second experimentation step

The second step concerns the comparison of performance achieved against specifically designed techniques tackling the tuning of parameters. In this work, we implemented classic SHO and CSA approaches assisted by IRace, named SHO + IRace and CSA + IRace. In this context, IRace includes off-line work, the process will be running and testing different samples of configurations, thus, the output corresponds to the statistical best possible values to run.

Tables 15 and 16 illustrates the results yielded by both approaches, where column F is the test function, and Opt presents the global optimum of the function. Column Best, Worst, and Avg are the given values for best value reached, worst value reached, the mean value, and the Avg time achieved in 30 executions.

Regarding LBMB-SHO vs SHO + IRace, the solutions achieved present almost an equal performance. For instance, both implementations reach the same amount of optimum values reached for column Best. However, the LBMB-SHO keeps a lead concerning the Avg and Avg time, and in the instances where the optimum (Opt) values are not reached, the distance is close. In this context, the proposed approach yields comparable results while significantly improving the computation time employed. For instance, the difference in the reported resources employed goes from 2 to 10 times, such as the one observed in f_5 and f_6 where the difference is more than 10 times. After applying Mann–Whitney, LBMB-SHO keeps a slightly difference compared to SHO + IRace in functions $f_4, f_5, f_9, f_{19}, f_{20}$, and f_{22}. In this context, the differences in performance are statistically significant (p-values $<$ 0.05), thus, LBMB-SHO is statistically superior in those functions.

Regarding LBMB-CSA vs CSA + IRace, the phenomenon observed is similar to the one presented in LBMB-SHO vs SHO + IRace; the tuned approaches by IRace can achieve a comparable performance. However, we highlight the fact that the version employing LBMB got more optimum values (Best) and the impact in the control of the termination criteria display great contribution in the overall performance. Also, after applying Mann–Whitney test, the results achieved can be described as LBMB-CSA being

Table 16
Results comparison of CSA+IRace vs. LBMB-CSA.

F	Opt	CSA+IRace					LBMB-CSA				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
f_1	0	**0.0000**	1.2227×10^{-13}	2.8883×10^{-14}	3.9880×10^{-14}	35.3093	**0.0000**	**0.0000**	**0.0000**	**0.0000**	18.2838
f_2	0	3.1973×10^{-9}	8.4505×10^{-6}	1.5476×10^{-6}	2.4237×10^{-6}	74.6915	1.7880×10^{-9}	7.8133×10^{-8}	1.9634×10^{-8}	2.1923×10^{-8}	30.3259
f_3	0	1.0115×10^{-12}	6.0013×10^{-9}	9.7726×10^{-10}	1.7846×10^{-9}	565.7959	**0.0000**	2.1526×10^{-11}	2.8536×10^{-12}	6.4462×10^{-12}	214.4357
f_4	0	2.0904×10^{-17}	5.8337×10^{-11}	8.2907×10^{-12}	1.7046×10^{-11}	80.9539	**0.0000**	1.7850×10^{-13}	3.7487×10^{-14}	5.7752×10^{-14}	31.4421
f_5	−12.569.49	**−12 569.487**	**−12 569.487**	**−12 569.487**	**0.0000**	62.6848	**−12 569.487**	**−12 569.487**	**−12 569.487**	**0.0000**	21.1642
f_6	0	**0.0000**	3.4674×10^{-11}	6.4915×10^{-12}	1.2774×10^{-11}	64.8181	**0.0000**	**0.0000**	**0.0000**	**0.0000**	26.7478
f_7	0	5.6963×10^{-9}	2.8457×10^{-7}	8.3645×10^{-8}	8.2887×10^{-8}	95.5441	6.3877×10^{-10}	2.3328×10^{-7}	3.2654×10^{-8}	6.7535×10^{-8}	37.2499
f_8	0	**0.0000**	3.2142×10^{-11}	8.0159×10^{-12}	1.1494×10^{-11}	82.5401	**0.0000**	5.7288×10^{-14}	1.3745×10^{-14}	1.8670×10^{-14}	33.5910
f_9	0	1.8286	1.8297	1.8287	0.0003	558.7875	1.8286	1.8286	1.8286	5.5733×10^{-16}	200.1375
f_{10}	1	**1.0000**	**1.0000**	**1.0000**	**0.0000**	363.3461	**1.0000**	**1.0000**	**1.0000**	**0.0000**	88.3443
f_{11}	−1.0316285	0.0000	0.0000	0.0000	0.0000	52.6489	0.0000	0.0000	0.0000	0.0000	14.8950
f_{12}	0.397887	−1.0312	−1.0130	−1.0267	0.0054	68.4265	−1.0315	−0.9978	−1.0235	0.0103	26.1120
f_{13}	3	32.6845	32.6845	32.6845	3.8918×10^{-15}	44.7113	32.6845	32.6845	32.6845	3.8918×10^{-15}	21.2859
f_{14}	−3.86	−2.0081	−2.0081	−2.0081	1.4043×10^{-16}	88.9599	−2.0081	−2.0081	−2.0081	0.0000	28.2264
f_{15}	−3.32	−1.1676	−1.1676	−1.1676	5.2074×10^{-16}	103.6407	−1.1676	−1.1676	−1.1676	1.8578×10^{-16}	39.0114
f_{16}	0	4.1298×10^{-15}	8.5590×10^{-12}	1.3649×10^{-12}	2.6452×10^{-12}	66.1485	**0.0000**	**0.0000**	**0.0000**	**0.0000**	26.9871
f_{17}	0	1.1294×10^{-15}	2.3778×10^{-10}	3.1313×10^{-11}	6.9811×10^{-11}	485.6491	**0.0000**	4.4918×10^{-13}	4.8795×10^{-14}	1.3362×10^{-13}	159.1209
f_{18}	unkown	0.0015	0.0015	0.0015	1.1791×10^{-13}	513.3789	0.0015	0.0015	0.0015	1.8814×10^{-15}	222.4313
f_{19}	0	4.2498×10^{-11}	5.4681×10^{-6}	8.2398×10^{-7}	1.6092×10^{-6}	603.6071	2.1839×10^{-14}	8.3235×10^{-9}	1.6347×10^{-9}	2.8330×10^{-9}	171.0849
f_{20}	0	2.9235×10^{-18}	7.5895×10^{-13}	1.4094×10^{-13}	2.6688×10^{-13}	85.54803	**0.0000**	**0.0000**	**0.0000**	**0.0000**	35.0854
f_{21}	0	0.4602	0.4602	0.4602	1.5777×10^{-12}	632.5531	0.4602	0.4602	0.4602	6.6844×10^{-16}	342.6311
f_{22}	0	4.5984×10^{-6}	0.0048	0.0008	0.0014	719.2438	1.5123×10^{-6}	0.0001	2.5648×10^{-5}	3.2096×10^{-5}	57.2893
f_{23}	0	1.0672×10^{-6}	1.0672×10^{-6}	1.0672×10^{-6}	5.8416×10^{-14}	167.0883	1.0672×10^{-6}	1.0672×10^{-6}	1.0672×10^{-6}	1.0278×10^{-16}	57.1890
f_{24}	0	1.1979×10^{-8}	4.9263×10^{-5}	9.0179×10^{-6}	1.5732×10^{-5}	119.12609	1.4000×10^{-8}	0.0001	4.4170×10^{-5}	5.1647×10^{-5}	50.4628

Table 17
Results comparison of SHO vs. LBMB-SHO.

F	Opt	SHO					LBMB-SHO				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
f_1	0	**0.0000**	0.0021	0.0001	0.0005	127.5526	**0.0000**	**0.0000**	**0.0000**	**0.0000**	18.6433
f_2	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	124.1584	**0.0000**	**0.0000**	**0.0000**	**0.0000**	33.4892
f_3	0	**0.0000**	0.0009	0.0007	0.0005	432.2254	**0.0000**	**0.0000**	**0.0000**	**0.0000**	177.5099
f_4	0	2.0157	6.4415	3.2551	0.1552	98.5002	0.0668	55.6351	5.1260	7.1011	25.0293
f_5	−12,569.487	-0.2664×10^4	-0.2557×10^4	-0.1687×10^4	0.7590	188.8100	-1.2692×10^4	15.2518	-5.8222×10^3	8.1121	17.6527
f_6	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	195.1140	**0.0000**	**0.0000**	**0.0000**	**0.0000**	20.1004
f_7	0	3.0010	3.9664	3.7945	0.4880	233.1744	4.4408×10^{-16}	0.9341	0.1000	1.4107	28.6423
f_8	0	**0.0000**	**0.0000**	**0.0000**	**0.0000**	213.5941	**0.0000**	**0.0000**	**0.0000**	**0.0000**	22.6069
f_9	0	40.4451	49.0097	47.7153	25.5103	355.0001	2.3498	73.8897	16.2587	13.8186	170.3205
f_{10}	1	9.1141	13.1067	11.6795	2.3341	367.8771	0.0033	11.7187	6.3874	5.1618	27.2279
f_{11}	−1.0316285	2.0994×10^{-17}	3.9885×10^{-17}	3.1195×10^{-17}	0.0991×10^{-17}	259.7845	6.3595×10^{-17}	4.5194	2.0290×10^{-17}	2.9159×10^{-17}	4.0946
f_{12}	0.397887	11.3134	16.5546	14.5414	5.1241	280.8997	0.3589	2.2683	0.4953	0.1777	4.2014
f_{13}	3	35.0011	35.8854	35.6896	0.0001	375.6333	29.6845	55.8719	32.6846	4.0269×10^{-5}	5.3021
f_{14}	−3.86	−2.5201	−1.0011	−1.2011	5.5764×10^{-7}	118.8870	−2.0081	1.9282	−2.0080	1.4829×10^{-8}	11.7642
f_{15}	−3.32	−1.2186	−0.0014	−0.3646	4.5465×10^{-7}	276.5471	−1.1676	2.1524	−1.1676	2.2514×10^{-9}	15.2136
f_{16}	0	6.8236×10^{-8}	79.2246	31.6767	38.7959	57.2111	1.0361×10^{-8}	75.6147	28.8705	38.1926	21.4518
f_{17}	0	**0.0000**	15.3113	5.2231	14 178.0000	100.6331	0.0000	0.0000	0.0000	0.0000	30.0628
f_{18}	unkown	0.0015	0.0498	0.0016	0.0001	147.6110	0.0015	0.0303	0.0016	0.0001	45.5892
f_{19}	0	0.0216	9.9276×10^{11}	6.4028×10^8	1.4747×10^9	85.6118	0.0011	1.3802×10^{16}	5.5042×10^{15}	1.1008×10^{16}	10.6033
f_{20}	0	2.1161×10^{-9}	22.7228	2.0126×10^{-6}	2.0643×10^{-6}	71.5552	1.4774×10^{-11}	2.6165	2.0903×10^{-7}	6.0620×10^{-7}	18.4379
f_{21}	0	0.4602	7.8173	0.8526	1.1771	466.0214	0.4602	22.1613	0.8526	1.1771	76.3696
f_{22}	0	0.4371	24 524.4178	2561.2954	7679.5683	93.7748	0.0322	2.6867×10^3	6.9827×10^3	1.1401×10^4	23.1293
f_{23}	0	1.0672×10^{-6}	0.0347	1.0699×10^{-6}	3.6735×10^{-9}	100.8874	1.0672×10^{-6}	0.0001	1.0679×10^{-6}	2.2910×10^{-9}	16.2799
f_{24}	0	**0.0000**	50.5197	6.4120×10^{-6}	3.4117×10^{-5}	112.5112	0.0000	19.4323	0.0012	0.0037	31.2105

Table 18
Results comparison of CSA vs. LBMB-CSA.

F	Opt	CSA					LBMB-CSA				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
f_1	0	3.9367×10^{-7}	0.0002	3.5578×10^{-5}	4.4307×10^{-5}	45.3535	**0.0000**	**0.0000**	**0.0000**	**0.0000**	18.2838
f_2	0	0.0011	0.0579	0.0229	0.0194	70.8322	1.7880×10^{-9}	7.8133×10^{-8}	1.9634×10^{-8}	2.1923×10^{-8}	30.3259
f_3	0	0.0026	3.1467	0.7338	0.9443	511.5846	**0.0000**	2.1526×10^{-11}	2.8536×10^{-12}	6.4462×10^{-12}	214.4357
f_4	0	0.0002	0.0239	0.0040	0.0073	79.3401	**0.0000**	1.7850×10^{-13}	3.7487×10^{-14}	5.7752×10^{-14}	31.4421
f_5	−12.569.487	**−12 569.487**	**−12 569.487**	**−12 569.487**	**0.0000**	60.7879	**−12 569.487**	**−12 569.487**	**−12 569.487**	**0.0000**	21.1642
f_6	0	1.7347×10^{-6}	0.0215	0.0047	0.0070	65.6590	**0.0000**	**0.0000**	**0.0000**	**0.0000**	26.7478
f_7	0	3.0854×10^{-5}	0.0458	0.0103	0.0127	91.2560	6.3877×10^{-10}	2.3328×10^{-7}	3.2654×10^{-8}	6.7535×10^{-8}	37.2499
f_8	0	5.9751×10^{-5}	0.0252	0.0043	0.0072	85.6089	**0.0000**	5.7288×10^{-14}	1.3745×10^{-14}	1.8670×10^{-14}	33.5910
f_9	0	1.8286	1.8286	1.8286	3.9234×10^{-13}	508.0567	1.8286	1.8286	1.8286	5.5733×10^{-16}	200.1375
f_{10}	1	**1.0000**	**1.0000**	**1.0000**	**0.0000**	243.2829	**1.0000**	**1.0000**	**1.0000**	**0.0000**	88.3443
f_{11}	−1.0316285	1.6310×10^{-11}	4.7419×10^{-7}	5.0198×10^{-8}	1.4141×10^{-7}	39.2778	0.0000	0.0000	0.0000	0.0000	14.8950
f_{12}	0.397887	−1.0310	−0.8411	−1.0063	0.0552	75.9839	−1.0315	−0.9978	−1.0235	0.0103	26.1120
f_{13}	3	32.6845	32.6845	32.6845	2.7080×10^{-6}	58.3441	32.6845	32.6845	32.6845	3.8918×10^{-15}	21.2859
f_{14}	−3.86	−2.0081	−2.0081	−2.0081	8.7328×10^{-9}	87.2376	−2.0081	−2.0081	−2.0081	0.0000	28.2264
f_{15}	−3.32	−1.1676	−1.1676	−1.1676	3.3236×10^{-9}	115.8929	−1.1676	−1.1676	−1.1676	1.8578×10^{-16}	39.0114
f_{16}	0	9.6475×10^{-5}	0.0029	0.0009	0.0009	66.9064	**0.0000**	**0.0000**	**0.0000**	**0.0000**	26.9871
f_{17}	0	1.6371×10^{-6}	0.0611	0.0130	0.0187	419.1573	**0.0000**	4.4918×10^{-13}	4.8795×10^{-14}	1.3362×10^{-13}	159.1209
f_{18}	unkown	0.0015	0.0034	0.0019	0.0006	526.6856	0.0015	0.0015	0.0015	1.8814×10^{-15}	222.4313
f_{19}	0	0.1162	65 038.3455	7523.7830	19 387.6540	411.6530	2.1839×10^{-14}	8.3235×10^{-9}	1.6347×10^{-9}	2.8330×10^{-9}	171.0849
f_{20}	0	1.4996×10^{-9}	0.0023	0.0003	0.0007	82.7588	**0.0000**	**0.0000**	**0.0000**	**0.0000**	35.0854
f_{21}	0	0.4602	0.4604	0.4603	7.0163×10^{-5}	843.2206	0.4602	0.4602	0.4602	6.6844×10^{-16}	342.6311
f_{22}	0	0.6120	35.1048	11.6549	11.3508	132.3475	1.5123×10^{-6}	0.0001	2.5648×10^{-5}	3.2096×10^{-5}	57.2893
f_{23}	0	1.0742×10^{-6}	2.4903×10^{-5}	7.4841×10^{-6}	7.9670×10^{-6}	137.2596	1.0672×10^{-6}	1.0672×10^{-6}	1.0672×10^{-6}	1.0278×10^{-16}	57.1890
f_{24}	0	1.0509×10^{-6}	0.0001	4.0433×10^{-5}	3.8803×10^{-5}	118.1662	1.4000×10^{-8}	0.0001	4.4170×10^{-5}	5.1647×10^{-5}	50.4628

statistically superior (p-values < 0.05) on functions $f_4, f_{16}, f_{19}, f_{20}$, and f_{23}.

Regarding the conclusion of this step, the usage of LBMB proved to be a competitive option against specifically designed methods to tackle the tuning of parameters. However, we observed some difficulties by LBMB searching the solution space (usually present in the StdDev values). Firstly, small range in the parameter space for the population size when IRace had a good performance working with 89 agents. In this context, the suitable size can vary for each algorithm, thus, this feature needs to evolve in order to make a more generic architecture auto-updating his values (for each scheme) on run-time. Nevertheless, in future works we consider a deeper comparison against multiple tuning methods in order to further test and prove the capabilities of our proposed on-line architecture.

4.4.3. Third experimentation step

In this third step, we present the comparison between the traditional optimization algorithms against LBMB-SHO and LBMB-CSA. The main objective is to demonstrate the improvements in the performance by the inclusion of learning/adaptability methods to a classic optimization tool.

Tables 17 and 18 illustrates the results yield by both approaches, where column F correspond to the test function solved, and Opt depicts the global optimum for the given function. Column Best, Worst, and Avg are the given values for best value reached, worst value reached, the mean value, and the Avg time achieved in 30 executions.

Regarding LBMB-SHO vs SHO, both implementation reaches the same amount of optimum (Best) values, in functions $f_1 - f_3$, f_6, and f_8. However, we highlight three facts observed in this comparison: LBMB achieved a close distance in the functions where the optimum was not reached, smaller values computed in the average (Avg), and smaller values computed as standard deviation (StdDev). Additionally, after applying Mann–Whitney test, the results achieved can be described as LBMB-SHO being statistically superior (p-values < 0.05) on functions f_4, f_7, f_9, f_{10}, $f_{12}, f_{17}, f_{19}, f_{20}, f_{22}$, and f_{24}, while SHO being never statistically superior (p-values ≥ 0.05). Regarding LBMB-CSA vs CSA, the

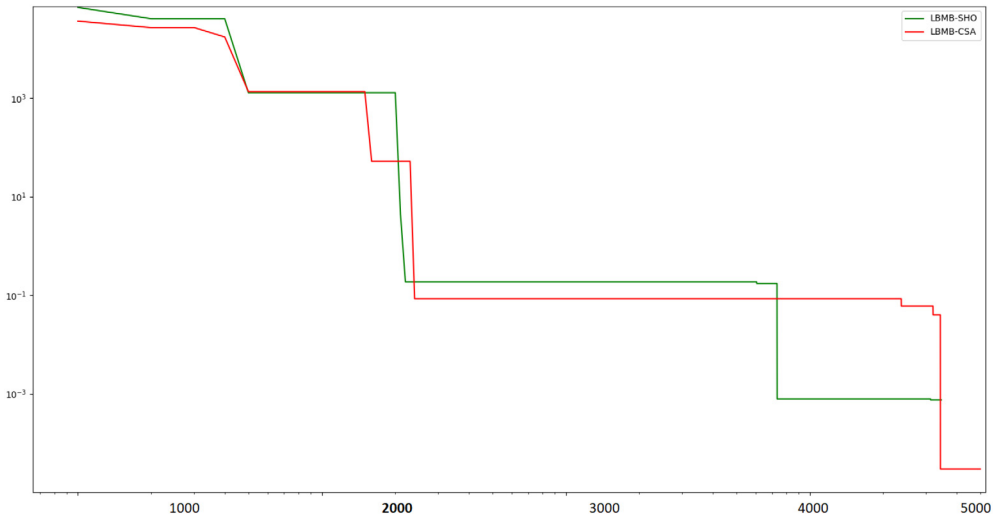


Fig. 2. Performance chart for function f_1 solved by LBMB-CHO and LBMB-CSA.

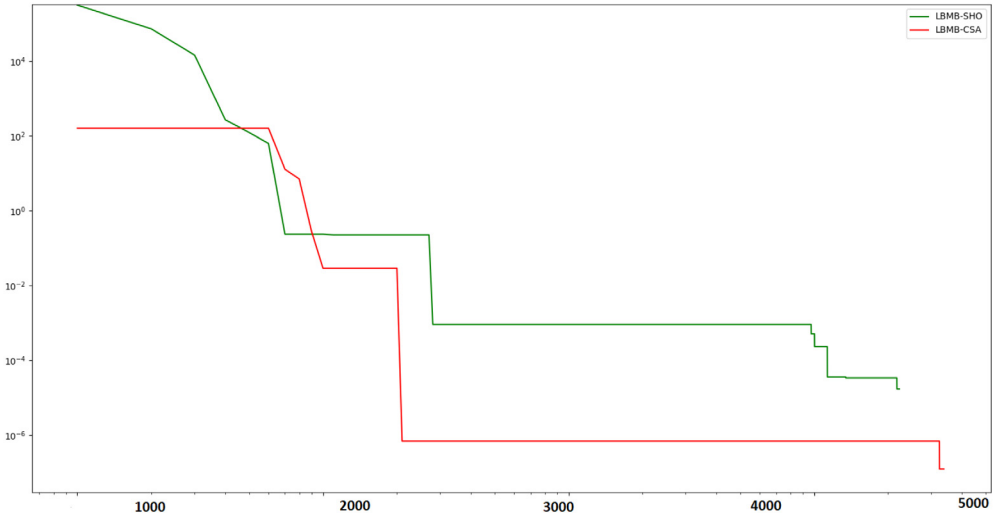


Fig. 3. Performance chart for function f_{11} solved by LBMB-CHO and LBMB-CSA.

proposed approach illustrates clear lead tackling the benchmark. For instance, more optimum values reached, smaller standard deviation (StdDev), and computation time (Avg time), the phenomenon observed is similar to the one presented for LBMB-SHO where the learning methods take great profit form the dynamic data generated through the search. Also, after applying Mann–Whitney test, the results achieved can be described as LBMB-CSA being statistically superior (p-values < 0.05) on functions f_4, f_5, f_8, f_{11}, f_{19}, and f_{22}. On the other hand, in Figs. 2, 3, and 4, we illustrate a convergence comparison between LBMB-CSA and LBMB-CHO solving the unimodal f_1, multimodal f_{11}, and high-dimensional f_{24} function. It is important to highlight the performance achieved which can be interpreted as robustness coupled with the smart behavior on the iterations performed.

Regarding the conclusion of this step, the employment of adaptiveness/learning methods improved the performance of the respective classic algorithm. Also, the usage of a learning-based termination criteria demonstrated to be a highly relevant component in order to optimize the computational time. The improvements in the resources employed while carrying out the search can be a key issue while tackling harder optimization problems.

4.4.4. Fourth experimentation step

In this subsection we illustrate the results achieved by the proposed approach solving the MKP. In Tables 19 and 20, the comparison between approaches implementing CSA are illustrated, we highlight in bold interesting results achieved. In both cases the over all performance is lead by LBMB-CSA reaching highest values for column Best which illustrates the best values achieved. After applying Mann–Whitney test, the results achieved can be described as LBMB-CSA being statistically superior (p-values < 0.05) on instances 5.100 (01, 02), 5.100.04, 5.250.04, 10.100 (00–04), and 10.250 (00–04) between LBMB-CSA and CSA in Table 19. Also, on instance 5.100 (00-04), 5.250 (02,04), 5.500 (00–03), 10.100.04, and 10.500 (00-04) between LBMB-CSA and CSA-IRace in Table 20. Nevertheless, the employment of the tool IRace, in 20, proved to successfully tackle a critical issue such as the fine-tuning of parameters, which ended up improving the over-all performance of the approach. On the other hand, we observed a big gap on the over all values computed for the column Avg time, LBMB-CSA computed a Σ value of 2400.03 against 38 038.84 and 38 033.06 for CSA and CSA-IRace respectively. In this regard, LBMB proves that the improvement of the

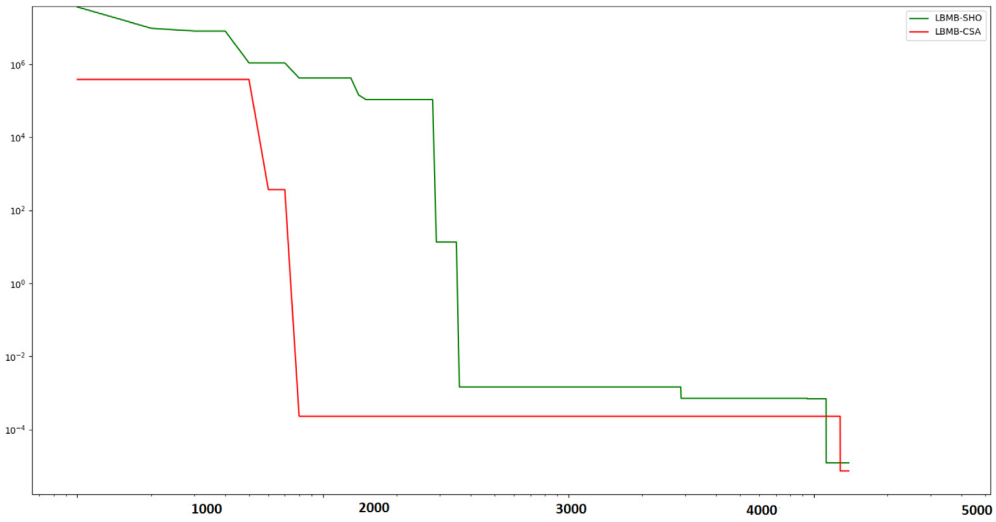


Fig. 4. Performance chart for function f_{24} solved by LBMB-CHO and LBMB-CSA.

Table 19
Computational results achieved by LBMB-CSA and CSA solving the MKP.

Instance	Opt	LBMB-CSA					CSA				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
5.100.00	24 381	23 772	20 040	20 883.73	467.56	11.58	23 798	20 690	20 975.74	442.95	91.36
5.100.01	24 274	23 904	19 934	20 506.52	421.50	11.20	23 593	19 889	20 230.07	472.09	103.33
5.100.02	23 551	22 571	20 368	21 458.24	309.27	18.56	23 316	20 029	22 007.04	424.48	84.66
5.100.03	23 534	22 796	20 350	21 041.05	481.21	9.58	22 752	20 107	21 248.42	450.01	100.64
5.100.04	23 991	23 796	20 566	20 906.63	448.55	11.21	22 602	20 121	20 902.71	517.87	103.36
5.250.00	59 312	58 413	50 120	51 338.22	675.85	59.26	58 801	50 808	55 354.28	705.69	500.67
5.250.01	61 472	61 065	51 769	52 482.72	890.04	59.81	61 036	51 476	57 179.24	758.26	545.63
5.250.02	62 130	60 214	53 390	55 542.76	1010.10	45.83	61 437	50 027	56 518.81	669.24	452.97
5.250.03	59 463	58 391	52 151	53 945.98	450.14	73.98	59 033	52 455	55 583.25	525.62	682.60
5.250.04	58 951	58 225	53 718	54 113.60	628.99	67.39	57 434	53 258	54 540.29	793.72	553.24
5.500.00	120 148	119 721	101 335	113 406.02	1124.28	231.80	119 539	101 941	116 458.41	1120.62	1669.70
5.500.01	117 879	117 636	103 863	110 319.81	900.72	247.76	116 922	104 714	113 985.85	828.37	1734.84
5.500.02	121 131	120 703	111 691	115 307.02	873.22	226.68	121 066	103 306	117 866.40	1129.54	2220.12
5.500.03	120 804	120 096	106 151	112 037.39	1079.61	260.35	120 598	101 859	118 796.66	894.15	2470.32
5.500.04	122 319	122 248	114 705	115 163.46	774.31	197.53	121 540	104 795	120 640.96	814.04	2684.56
10.100.00	23 064	22 648	19 962	20 848.84	328.52	4.39	22 823	20 291	21 054.37	336.36	145.00
10.100.01	22 801	22 438	20 084	20 650.36	197.50	7.33	21 995	19 878	20 971.24	337.19	215.51
10.100.02	22 131	21 375	18 602	20 488.50	383.30	5.71	22 012	19 973	20 884.02	368.46	168.03
10.100.03	22 772	22 088	20 021	20 881.95	411.82	5.54	22 242	19 293	20 274.07	401.99	150.22
10.100.04	22 751	22 196	19 015	21 285.91	387.99	5.39	22 692	19 151	20 941.84	320.86	207.03
10.250.00	59 187	58 591	50 598	51 230.07	728.49	32.35	58 365	50 430	56 412.77	573.14	1044.34
10.250.01	58 781	58 186	52 104	54 271.19	227.24	33.92	58 547	52 756	56 240.46	534.38	1013.60
10.250.02	58 097	57 759	49 269	52 173.73	836.34	28.64	57 821	49 226	55 257.98	619.79	941.95
10.250.03	61 000	60 550	51 348	55 649.28	745.56	34.50	59 800	51 940	55 701.36	677.03	1164.33
10.250.04	58 092	57 493	51 743	53 238.50	365.78	38.26	58 025	50 508	56 619.27	536.69	1262.05
10.500.00	117 821	117 362	100 529	106 512.01	950.69	134.54	117 273	101 707	110 671.37	888.96	3271.74
10.500.01	119 249	119 147	97 762	100 502.26	1606.21	128.67	118 798	100 283	110 872.52	1058.69	3600.26
10.500.02	119 215	118 892	101 154	109 578.89	533.71	140.66	118 261	100 690	110 768.08	734.35	3963.56
10.500.03	118 829	118 082	98 770	102 774.43	1027.23	115.91	118 060	99 409	110 574.93	873.92	3350.48
10.500.04	116 530	116 322	100 411	103 472.26	630.34	151.71	116 091	100 552	110 962.59	736.73	3542.75
Σ	2 023 660	**2 006 680**	**1 751 523**	1 832 011.34	19 896.09	**2400.03**	1 890 181	1 731 562	**1 910 495.00**	19 545.19	38 038.84

termination criteria critically improved the usage of computational resources.

In Tables 21 and 22 the comparison between approaches implementing SHO are illustrated, we highlight in bold the interesting results achieved. In both cases the overall performance is led by LBMB-SHO reaching highest values for column Best which illustrates the best values achieved. After applying Mann–Whitney test, the results achieved can be described as LBMB-CSA being statistically superior (p-values < 0.05) on instances 5.100 (00,03,04), 5.250.03, 5.500.01, 10.100 (00–03), 10.250 (00–02), and 20.500 (00,02,04) between LBMB-SHO and SHO in Table 21. Also, on instance 5.250.00, 5.500 (00,02,03,04), 10.100 (00,01), 10.250.04, 10.500 (00,02,04) between LBMB-SHO and SHO-IRace in Table 22. In this regard, an equal phenomenon was observed regarding the employment of IRace and the average of solving time. This experimentation step gave us new ideas regarding actual applications on real-life problems. The proposed approach was able to achieve close to the optimum results values which we identify as having great potential to keep working on in future proposals.

5. Conclusion and future work

In this work, a learning-based modular architecture was proposed in order to solve optimization problems. This architecture was based on a population-based strategy in order to search the

Table 20
Computational results achieved by LBMB-CSA and CSA + IRace solving the MKP.

Instance	Opt	LBMB-CSA					CSA + IRace				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
5.100.00	24 381	23 772	20 040	20 883.73	467.56	11.58	23 226	16 612	19 922.93	479.45	107.26
5.100.01	24 274	23 904	19 934	20 506.52	421.50	11.20	23 861	16 736	19 660.91	299.37	143.21
5.100.02	23 551	22 571	20 368	21 458.24	309.27	18.56	21 687	15 790	18 585.92	465.72	115.02
5.100.03	23 534	22 796	20 350	21 041.05	481.21	9.58	23 477	17 014	19 916.13	437.07	111.61
5.100.04	23 991	23 796	20 566	20 906.63	448.55	11.21	23 436	16 220	19 415.35	470.92	148.96
5.250.00	59 312	58 413	50 120	51 338.22	675.85	59.26	59 084	43 509	52 298.38	818.10	529.22
5.250.01	61 472	61 065	51 769	52 482.72	890.04	59.81	61 311	44 235	52 228.75	375.49	558.49
5.250.02	62 130	60 214	53 390	55 542.76	1010.10	45.83	61 759	42 162	50 882.47	816.64	507.88
5.250.03	59 463	58 391	52 151	53 945.98	450.14	73.98	58 867	44 441	53 465.08	1017.30	628.22
5.250.04	58 951	58 225	53 718	54 113.60	628.99	67.39	58 613	44 629	52 296.44	450.88	666.75
5.500.00	120 148	119 721	101 335	113 406.02	1124.28	231.80	119 933	91 011	102 714.97	1117.43	1940.04
5.500.01	117 879	117 636	103 863	110 319.81	900.72	247.76	116 037	87 534	101 246.10	1889.82	1824.36
5.500.02	121 131	120 703	111 691	115 307.02	873.22	226.68	120 996	90 619	102 642.96	632.50	2723.78
5.500.03	120 804	120 096	106 151	112 037.39	1079.61	260.35	120 142	88 641	100 649.99	1187.26	1913.90
5.500.04	122 319	122 248	114 705	115 163.46	774.31	197.53	121 725	89 866	101 515.29	762.84	2593.77
10.100.00	23 064	22 648	19 962	20 848.84	328.52	4.39	22 483	17 301	21 143.43	685.54	175.40
10.100.01	22 801	22 438	20 084	20 650.36	197.50	7.33	21 976	17 936	21 019.76	541.37	160.18
10.100.02	22 131	21 375	18 602	20 488.50	383.30	5.71	21 927	17 339	19 950.85	377.15	164.74
10.100.03	22 772	22 088	20 021	20 881.95	411.82	5.54	22 116	17 613	20 601.30	583.59	179.76
10.100.04	22 751	22 196	19 015	21 285.91	387.99	5.39	21 609	17 570	20 455.41	506.71	151.85
10.250.00	59 187	58 591	50 598	51 230.07	728.49	32.35	58 694	42 764	54 739.64	859.40	827.90
10.250.01	58 781	58 186	52 104	54 271.19	227.24	33.92	57 958	42 615	55 792.02	1116.87	1041.91
10.250.02	58 097	57 759	49 269	52 173.73	836.34	28.64	56 844	43 681	55 622.51	1085.26	810.52
10.250.03	61 000	60 550	51 348	55 649.28	745.56	34.50	60 546	44 757	55 701.34	563.88	1178.74
10.250.04	58 092	57 493	51 743	53 238.50	365.78	38.26	57 209	42 727	54 005.66	741.00	869.87
10.500.00	117 821	117 362	100 529	106 512.01	950.69	134.54	117 744	90 809	103 545.73	1141.84	3194.63
10.500.01	119 249	119 147	97 762	100 502.26	1606.21	128.67	118 529	88 915	100 799.28	1064.60	3799.31
10.500.02	119 215	118 892	101 154	109 578.89	533.71	140.66	118 408	92 658	104 744.63	1293.48	3626.80
10.500.03	118 829	118 082	98 770	102 774.43	1027.23	115.91	117 975	89 565	102 062.23	1407.37	4163.41
10.500.04	116 530	116 322	100 411	103 472.26	630.34	151.71	116 069	93 531	105 259.29	924.07	3175.59
Σ	2 023 660	**2 006 680**	**1 751 523**	**1 832 011.34**	19 896.09	**2400.03**	2 004 241	1 508 800	1 762 884.75	24 112.92	38 033.06

Table 21
Computational results achieved by LBMB-SHO and SHO solving the MKP.

Instance	Opt	LBMB-SHO					SHO				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
5.100.00	24 381	24 006	18 545	20 617.70	1003.73	28.62	23 473	17 605	18 326.66	825.35	188.32
5.100.01	24 274	23 444	18 017	20 019.91	921.35	23.81	23 771	17 160	19 987.47	932.06	162.52
5.100.02	23 551	23 495	17 273	19 126.97	899.89	16.63	22 869	15 153	18 303.68	1301.52	162.44
5.100.03	23 534	23 093	17 543	20 318.19	1096.65	29.84	22 389	16 444	17 634.68	1287.12	204.67
5.100.04	23 991	23 878	17 381	20 116.50	984.42	19.30	22 957	15 942	18 907.14	1057.08	187.61
5.250.00	59 312	58 833	48 598	56 603.54	1213.15	171.26	58 917	50 111	55 679.91	1626.48	1307.64
5.250.01	61 472	59 928	46 693	55 828.59	1585.51	150.17	60 928	47 116	52 242.47	1876.38	865.73
5.250.02	62 130	60 746	45 360	55 844.66	1770.15	154.72	60 771	54 811	57 816.65	1762.32	965.67
5.250.03	59 463	58 718	48 739	56 928.70	1586.45	155.00	58 038	48 313	54 731.97	1668.99	1438.75
5.250.04	58 951	58 406	46 294	56 040.19	1467.95	155.21	58 414	49 955	56 358.30	1555.86	1288.49
5.500.00	120 148	120 011	102 884	113 311.47	2154.46	453.64	119 213	112 484	114 587.61	2557.73	5799.69
5.500.01	117 879	116 990	110 393	112 148.99	2749.68	568.89	116 776	105 084	108 724.57	2979.51	4386.95
5.500.02	121 131	120 353	109 087	111 479.48	2070.32	611.39	120 378	110 540	113 635.82	2673.51	4747.46
5.500.03	120 804	119 794	105 162	111 370.81	2430.86	633.04	120 066	106 001	111 945.72	2505.05	3793.43
5.500.04	122 319	121 328	101 895	109 446.76	2268.18	619.95	120 328	112 426	116 248.75	2540.73	4763.31
10.100.00	23 064	22 135	18 502	21 644.80	1151.77	12.89	22 816	18 975	20 143.89	1419.68	135.75
10.100.01	22 801	21 874	17 605	20 872.56	1284.38	15.81	21 554	17 997	19 214.20	1294.27	111.37
10.100.02	22 131	21 851	16 944	20 205.61	1153.66	17.51	21 426	16 810	19 732.66	1094.17	96.94
10.100.03	22 772	22 410	19 173	21 372.60	1061.43	11.27	22 255	18 818	20 245.79	1180.65	87.90
10.100.04	22 751	22 068	18 610	20 901.89	1134.44	11.53	22 299	19 243	21 217.86	1573.70	110.04
10.250.00	59 187	59 026	52 424	55 166.92	1623.68	83.96	58 697	49 263	53 923.89	1814.22	742.09
10.250.01	58 781	57 850	50 183	56 455.44	1996.78	72.19	56 598	48 050	54 454.42	2137.42	727.58
10.250.02	58 097	57 536	54 077	56 208.44	1549.30	82.66	57 536	49 356	55 524.26	1735.09	561.73
10.250.03	61 000	60 514	52 741	56 006.59	1692.53	98.92	60 623	52 564	56 372.57	2189.05	650.49
10.250.04	58 092	57 495	50 591	54 386.27	1390.96	129.89	57 371	49 010	54 984.89	2002.65	917.14
10.500.00	117 821	117 190	100 438	112 496.16	2444.55	399.08	117 153	105 507	110 926.67	2914.91	2163.26
10.500.01	119 249	118 593	100 427	111 621.21	1799.25	402.99	118 524	104 649	110 609.87	2148.25	3148.82
10.500.02	119 215	118 173	102 482	113 513.50	2404.89	391.68	116 880	108 441	112 864.85	1978.30	2635.62
10.500.03	118 829	117 996	100 927	112 404.59	2183.59	284.58	117 331	111 511	115 427.58	3343.33	3239.17
10.500.04	116 530	116 121	101 890	114 881.75	2403.50	341.61	116 456	106 287	111 184.83	2461.31	3514.57
Σ	2 023 660	**2 003 855**	1 710 878	**1 887 341**	49 477	**6148**	1 996 807	**1 755 626**	1 871 960	56 437	49 105

Table 22
Computational results achieved by LBMB-SHO and SHO + IRace solving the MKP.

Instance	Opt	LBMB-SHO					SHO + IRace				
		Best	Worst	Avg	StdDev	Avg time (s)	Best	Worst	Avg	StdDev	Avg time (s)
5.100.00	24 381	24 006	18 545	20 617.70	1003.73	28.62	24 116	17 781	20 282.24	697.13	227.46
5.100.01	24 274	23 444	18 017	20 019.91	921.35	23.81	24 029	17 884	22 872.88	734.75	194.60
5.100.02	23 551	23 495	17 273	19 126.97	899.89	16.63	22 886	16 733	18 235.29	833.65	184.89
5.100.03	23 534	23 093	17 543	20 318.19	1096.65	29.84	22 369	17 959	21 391.76	942.75	190.81
5.100.04	23 991	23 878	17 381	20 116.50	984.42	19.30	22 886	17 604	19 286.47	982.87	153.86
5.250.00	59 312	58 833	48 598	56 603.54	1213.15	171.26	56 787	51 576	54 343.93	1234.45	1238.19
5.250.01	61 472	59 928	46 693	55 828.59	1585.51	150.17	60 928	52 737	56 515.70	1291.16	1257.33
5.250.02	62 130	60 746	45 360	55 844.66	1770.15	154.72	61 647	51 814	55 002.68	1493.17	1020.08
5.250.03	59 463	58 718	48 739	56 928.70	1586.45	155.00	59 038	53 977	56 865.78	1227.93	1113.80
5.250.04	58 951	58 406	46 294	56 040.19	1467.95	155.21	58 804	52 952	54 295.98	1260.86	1216.80
5.500.00	120 148	120 011	102 884	113 311.47	2154.46	453.64	118 915	102 884	110 919.93	1898.16	3878.19
5.500.01	117 879	116 990	110 393	112 148.99	2749.68	568.89	116 633	100 393	112 126.83	1859.30	5941.32
5.500.02	121 131	120 353	109 087	111 479.48	2070.32	611.39	120 079	100 871	103 667.55	1990.14	4993.24
5.500.03	120 804	119 794	105 162	111 370.81	2430.86	633.04	118 445	101 629	108 195.38	2162.66	4013.79
5.500.04	122 319	121 328	101 895	109 446.76	2268.18	619.95	120 949	98 959	111 337.97	2159.02	5169.46
10.100.00	23 064	22 135	18 502	21 644.80	1151.77	12.89	22 356	19 023	20 638.05	1075.01	146.71
10.100.01	22 801	21 874	17 605	20 872.56	1284.38	15.81	21 988	18 415	19 952.58	787.22	128.09
10.100.02	22 131	21 851	16 944	20 205.61	1153.66	17.51	21 769	18 129	20 313.03	915.28	105.95
10.100.03	22 772	22 410	19 173	21 372.60	1061.43	11.27	22 173	19 495	21 290.30	820.49	97.92
10.100.04	22 751	22 068	18 610	20 901.89	1134.44	11.53	21 806	18 528	20 123.93	1078.09	130.36
10.250.00	59 187	59 026	52 424	55 166.92	1623.68	83.96	58 506	50 085	56 130.48	1211.57	624.13
10.250.01	58 781	57 850	50 183	56 455.44	1996.78	72.19	58 331	47 748	57 110.16	1907.34	645.84
10.250.02	58 097	57 536	54 077	56 208.44	1549.30	82.66	57 536	49 652	54 384.35	1595.75	721.10
10.250.03	61 000	60 514	52 741	56 006.59	1692.53	98.92	60 040	50 641	58 471.39	1554.68	656.30
10.250.04	58 092	57 495	50 591	54 386.27	1390.96	129.89	57 280	49 349	52 603.81	1534.55	849.42
10.500.00	117 821	117 190	100 438	112 496.16	2444.55	399.08	117 378	102 905	106 429.26	1874.86	2925.52
10.500.01	119 249	118 593	100 427	111 621.21	1799.25	402.99	117 782	101 786	112 017.07	1826.43	4019.78
10.500.02	119 215	118 173	102 482	113 513.50	2404.89	391.68	119 083	104 599	107 647.32	1327.66	3226.79
10.500.03	118 829	117 996	100 927	112 404.59	2183.59	284.58	117 747	102 726	103 191.87	2011.26	2665.42
10.500.04	116 530	116 121	101 890	114 881.75	2403.50	341.61	114 826	100 861	110 272.41	1777.25	3024.56
Σ	2 023 660	**2 003 855**	**1 710 878**	**1 887 341**	49 477	**6148**	1 997 112	1 709 695	1 845 916.38	42 065.43	50 761.69

solution space, thus, spotted hyena optimizer and crow search algorithms were employed to carry out this objective. Regarding the focus of LBMB, two major issues on a population-based method were tackled, a smart balance behind the population size and termination criteria. The balance given by the learning model was based on multiple linear regression which influenced the architecture to give priority (while carrying out the search) to more suited sizes to perform at certain times and at what time was suited to terminate the search. Regarding the performance achieved through the experimentation phase, promising results were obtained in this first attempt designing LBMB. The versions LBMB-SHO and LBMB-CSA achieved a comparable performance against the algorithms supported by IRace, traditional metaheuristics implementation, and SOTA methods. The overall performance observed depicted that LBMB methods were slightly superior in certain functions and MKP instances in the results reported. However, limitations were observed in the configured values for each scheme (population size): static and a small parameter space defined. In this context, a first answer to be proposed includes a self-control mechanism for the scheme values, where they self-update their values on run-time taking into account the diversification and intensification of the parameter space for the population size and the most suitable amount to perform at certain times in the search. Additionally, the reported average computational time has a huge impact, it is well-known how valuable as a resource the computational time can be, thus, giving us room for future improvements and higher profit exploiting learning models to optimize the solving process. Concerning the computational complexity, two issues can be described: learning technique and population-based method employed. The linear model employed has low implementation complexity, linear regression has been widely used in several areas and to date has proved to be effective. Nevertheless, the issue of concern is the definition of the independent variables and their computational cost in the implementation. Also, the complexity in this field can potentially increase if we add multiple learning methods such as deep learning and reinforcement learning methods. In this regard, an algorithm selector would be needed in order to control the parallel execution and performance. Moreover, even if more complex methods are implemented in the architecture, improvements in the performance will not be a fact and tuning such methods can be an arduous task.

Regarding the future works, it was illustrated in the contributions that the key word was scalability. The objective is to exploit multiple techniques in order to further improve the proposed modules from the architecture. For instance, new regression techniques for modules 3 and 4, where the most fitted technique to be applied in the data is adequately selected. Also, the possibilities to try multiple population-based algorithms working in parallel is an actual option. Lastly, the performance achieved by solving MKP instances concluded on new ideas and motivations to keep working in the proposed approach. Thus, we aim to design a more mature LBMB with the capabilities to tackle several discrete optimization problems, such as Manufacturing Cell Design Problem (MCDP), Set Covering Problem (SCP), and real-time application. In this regard, this architecture can be a surprise to the optimization field thanks to his on-line nature, where the key is the adaptability in the search through unknown environments. However, in order to achieve this, scalability and evolution are the focus to be tackled in future projects.

CRediT authorship contribution statement

Emanuel Vega: Formal analysis, Investigation, Resource, Software, Writing – original draft, Writing – review & editing. **Ricardo Soto:** Formal analysis, Investigation, Resource, Writing – original draft, Writing – review & editing. **Broderick Crawford:** Investigation, Validation, Writing – original draft. **Javier Peña:** Formal

analysis, Investigation, Software. **Pablo Contreras:** Investigation, Software. **Carlos Castro:** Investigation, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

Ricardo Soto is supported by Grant CONICYT/FONDECYT/REGULAR/1190129. Broderick Crawford is supported by Grant ANID/FONDECYT/REGULAR/1210810, and Emanuel Vega is supported by Grant ANID/Scholarship Program/DOCTORADO, Chile NACIONAL/2020-21202527. All authors approved the version of the manuscript to be published.

References

[1] K. Hussain, M.N.M. Salleh, S. Cheng, Y. Shi, Metaheuristic research: a comprehensive survey, Artif. Intell. Rev. 52 (4) (2019) 2191–2233.
[2] M. Abdel-Basset, L. Abdel-Fatah, A.K. Sangaiah, Metaheuristic algorithms: A comprehensive review, in: Computational Intelligence for Multimedia Big Data on the Cloud with Engineering Applications, 2018, pp. 185–231.
[3] H. Song, I. Triguero, E. Ozcan, A review on the self and dual interactions between machine learning and optimisation, Prog. Artif. Intell. 8 (2) (2019) 143–165.
[4] L. Dioşan, M. Oltean, Evolutionary design of evolutionary algorithms, Genet. Program. Evol. Mach. 10 (3) (2009) 263–306.
[5] E.R. Hruschka, R.J. Campello, A.A. Freitas, A survey of evolutionary algorithms for clustering, IEEE Trans. Syst. Man Cybern. C 39 (2) (2009) 133–155.
[6] S. Thrun, L. Pratt (Eds.), Learning to Learn, Springer Science & Business Media, 2012.
[7] P.B. Brazdil, C. Soares, J.P. Da Costa, Ranking learning algorithms: using ibl and meta-learning on accuracy and time results, Mach. Learn. 50 (3) (2003) 251–277.
[8] S. Asta, E. Özcan, T. Curtois, A tensor based hyper-heuristic for nurse rostering, Knowl. Based Syst. 98 (2016) 185–199.
[9] G. Dhiman, V. Kumar, Spotted hyena optimizer: A novel bio-inspired based metaheuristic technique for engineering applications, Adv. Eng. Softw. 114 (2017) 48–70.
[10] A. Askarzadeh, A novel metaheuristic method for solving constrained engineering optimization problems: crow search algorithm, Comput. Struct. 169 (2016) 1–12.
[11] Q. Luo, J. Li, Y. Zhou, L. Liao, Using spotted hyena optimizer for training feedforward neural networks, Cogn. Syst. Res. 65 (2021) 1–16.
[12] R. Soto, B. Crawford, E. Vega, A. Gómez, J.A. Gómez-Pulido, Solving the set covering problem using spotted hyena optimizer and autonomous search, in: F. Wotawa, G. Friedrich, I. Pill, R. Koitz-Hristov, M. Ali (Eds.), Advances and Trends in Artificial Intelligence. From Theory to Practice. IEA/AIE 2019, in: Lecture Notes in Computer Science, vol. 11606, Springer, Cham, 2019.
[13] E. Vega, R. Soto, B. Crawford, J. Peña, C. Castro, A learning-based hybrid framework for dynamic balancing of exploration-exploitation: Combining regression analysis and metaheuristics, Mathematics 1 (2021).
[14] Y. Meraihi, A.B. Gabis, A. Ramdane-Cherif, D. Acheli, A comprehensive survey of Crow Search Algorithm and its applications, Artif. Intell. Rev. 54 (4) (2021) 2669–2716.
[15] E.G. Talbi, Metaheuristics: From Design to Implementation, Wiley Publishing, ISBN: 0470278587, 2009, 9780470278581.
[16] E.G. Talbi, Machine learning into metaheuristics: A survey and taxonomy, ACM Comput. Surv. 54 (6) (2021) 1–32.
[17] L. Calvet, J. de Armas, D. Masip, A.A. Juan, Learnheuristics: hybridizing metaheuristics with machine learning for optimization with dynamic inputs, Open Math. 15 (1) (2017) 261–280.
[18] M.G.P. de Lacerda, L.F. de Araujo Pessoa, F.B. de Lima Neto, T.B. Ludermir, H. Kuchen, A systematic literature review on general parameter control for evolutionary and swarm-based algorithms, Swarm Evol. Comput. 60 (2021) 100777.
[19] B. Adenso-Diaz, M. Laguna, Fine-tuning of algorithms using fractional experimental designs and local search, Oper. Res. 54 (2006) 99–114.
[20] L. Diosan, M. Oltean, Evolutionary design of evolutionary algorithms, Genet. Program. Evol. Mach. 10 (3) (2009) 263–306.
[21] S. Asta, E. Özcan, T. Curtois, A tensor based hyper-heuristic for nurse rostering, Knowl. Based Syst. 98 (2016) 185–199.
[22] R. Soto, B. Crawford, F. González, E. Vega, C. Castro, F. Paredes, Solving the manufacturing cell design problem using human behaviorbased algorithm supported by autonomous search, IEEE Access 7 (2019) 132228–132239.
[23] J. Zhang, H. Chung, W.-L. Lo, Clustering-based adaptive crossover and mutation probabilities for genetic algorithms, IEEE Trans. Evol. Comput. 11 (3) (2007) 326–335.
[24] F. Streichert, G. Stein, H. Ulmer, A. Zell, A clustering based niching method for evolutionary algorithms, in: Genetic and Evolutionary Computation Conference, Springer, 2003, pp. 644–645.
[25] L. Santos, M.H. Ribeiro, A. Plastino, S.L. Martins, A hybrid GRASP with data mining for the maximum diversity problem, in: International Workshop on Hybrid Metaheuristics, Springer, 2005, pp. 116–127.
[26] M.H. Ribeiro, A. Plastino, S.L. Martins, Hybridization of GRASP metaheuristic with data mining techniques, J. Math. Model. Algorithms 5 (1) (2006) 23–41.
[27] D. Bhandari, C.A. Murthy, S.K. Pal, Genetic algorithms with elitist model and its convergence, Int. J. Pattern Recognit. Artif. Intell. 10 (6) (1996) 731–747.
[28] C. Murthy, D. Bhandari, S.K. Pal, E-optimal stopping time for genetic algorithms, Fund. Inform. 35 (1–4) (1998) 91–111.
[29] Y. Gajpal, P. Abad, An ant colony system (acs) for vehicle routing problem with simultaneous delivery and pickup, Comput. Oper. Res. 36 (12) (2009) 3215–3223, New developments on hub location.
[30] A.E. Rizzoli, A.V. Donati, R. Montemanni, L.M. Gambardella, Ant colony system for a dynamic vehicle routing problem, J. Comb. Optim. 10 (2005) 327–343.
[31] J.T. Tsai, T.K. Liu, J.H. Chou, Hybrid Taguchi-genetic algorithm for global numerical optimization, IEEE Trans. Evol. Comput. 8 (2004) 365–377.
[32] P. Romet, F. Gechter, E.H. Aglzim, O. Grunder, S.M. Senouci, An environment based approach for the ant colony convergence, Procedia Comput. Sci. 177 (2020) 488–495.
[33] B. Jain, H. Pohlheim, J. Wegener, On termination criteria of evolutionary algorithms, 2001, p. 768.
[34] M. Veluscek, T. Kalganova, P. Broomhead, Improving ant colony optimization performance through prediction of best termination condition, in: 2015 IEEE International Conference on Industrial Technology (ICIT), 2015, pp. 2394–2402.
[35] J. Digalakis, K. Margaritis, On benchmarking functions for genetic algorithms, Int. J. Comput. Math. 77 (4) (2001) 481–506.
[36] X. Yang, Firefly algorithm, stochastic test functions and design optimisation, Int. J. Bio-Inspired Comput. 2 (2) (2010) 78–84.
[37] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, Adv. Eng. Softw. 69 (2014) 46–61.
[38] F. Gao, L. Han, Implementing the Nelder–Mead simplex algorithm with adaptive parameters, Comput. Optim. Appl. 51 (1) (2012) 259–277.
[39] J.J. Moré, B.S. Garbow, K.E. Hillstrom, Testing unconstrained optimization software, ACM Trans. Math. Softw. 7 (1) (1981) 17–41.
[40] D. Pisinger, The quadratic knapsack problem—a survey, Discrete Appl. Math. 155 (5) (2007) 623–648.
[41] E. Horowitz, S. Sahni, Computing partitions with applications to the knapsack problem, J. ACM 21 (2) (1974) 277–292.
[42] S. Mirjalili, A. Lewis, S-shaped versus v-shaped transfer functions for binary particle swarm optimization, Swarm Evol. Comput. 9 (2013) 1–14.
[43] J.M. Lanza-Gutierrez, B. Crawford, R. Soto, N. Berrios, J.A. Gomez-Pulido, F. Paredes, Analyzing the effects of binarization techniques when solving the set covering problem through swarm optimization, Expert Syst. Appl. 70 (2017) 67–82.
[44] S. Mirjalili, A. Lewis, The whale optimization algorithm, Adv. Eng. Softw. 95 (2016) 51–67.
[45] E.M. Cortés-Toro, B. Crawford, J.A. Gómez-Pulido, R. Soto, J.M. Lanza-Gutiérrez, A new metaheuristic inspired by the vapour-liquid equilibrium for continuous optimization, Appl. Sci. 8 (2018) 2080.
[46] I. Fajfar, Á. Bürmen, J. Puhan, The Nelder–Mead simplex algorithm with perturbed centroid for high-dimensional function optimization, Optim. Lett. 13 (5) (2019) 1011–1025.
[47] J. Kennedy, R. Eberhart, Particle swarm optimization, in: Proceedings of the IEEE International Conference on Neural Networks, Perth, Australia, 27 November–1 December 1995, pp. 1942–1948.
[48] R. Storn, K. Price, Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces, J. Glob. Optim. 11 (1997) 341–359.

[49] E. Rashedi, H. Nezamabadi-pour, S. Saryazdi, GSA: A gravitational search algorithm, Inform. Sci. 179 (2009) 2232–2248.
[50] S. Mirjalili, A. Lewis, The whale optimization algorithm, Adv. Eng. Softw. 95 (2016) 51–67.
[51] E.M. Cortés-Toro, B. Crawford, J.A. Gómez-Pulido, R. Soto, J.M. Lanza-Gutiérrez, A new metaheuristic inspired by the vapour-liquid equilibrium for continuous optimization, Appl. Sci. 8 (2018) 2080.
[52] J. Xu, F. Yan, Hybrid Nelder–Mead algorithm and dragonfly algorithm for function optimization and the training of a multilayer perceptron, Arab. J. Sci. Eng. 44 (4) (2019) 3473–3487.
[53] M. Lopez-Ibanez, J. Dubois-Lacoste, L.P. Caceres, M. Birattari, T. Stutzle, The irace package: Iterated racing for automatic algorithm configuration, Oper. Res. Perspect. 3 (2016) 43–58.